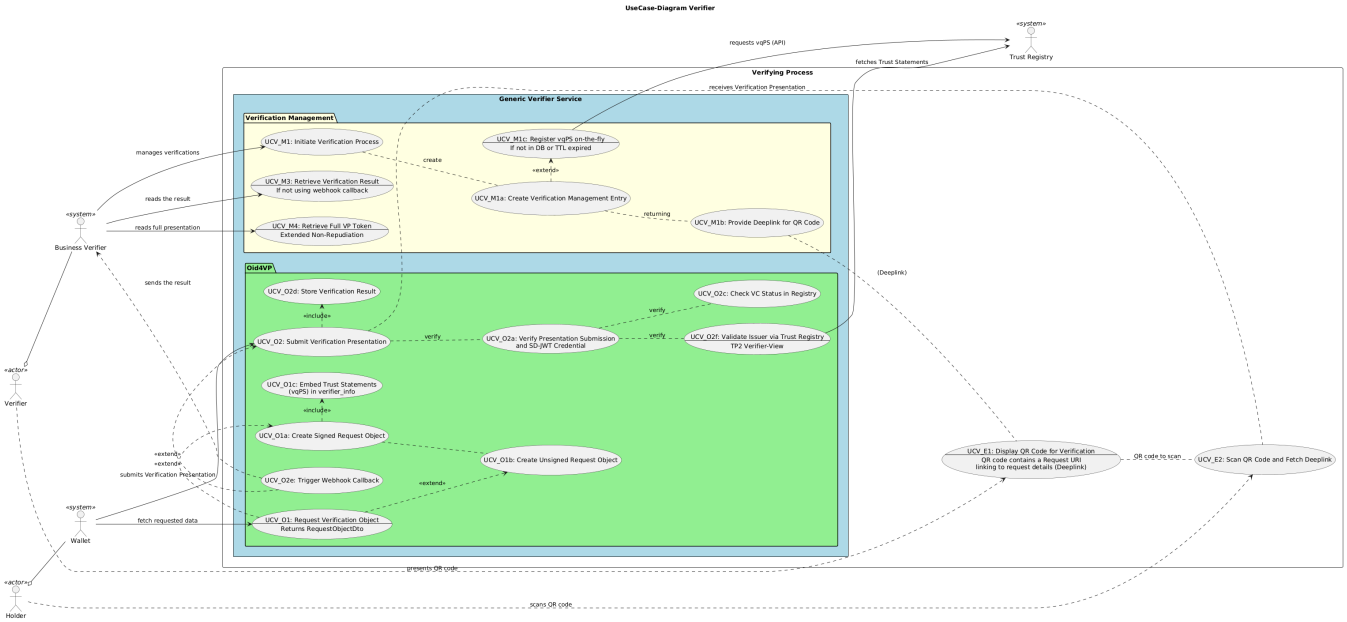
Verifier Documentation (pdf export)

- 1. Business Context
  - 1.1. Use Cases
  - 1.2. Actors
  - 1.3. External Systems
  - 1.4. Interactions
  - 1.5. Additional Information
- 2. Technical Context
  - 2.1. Core System
  - 2.2. Context
- 3. Building Block View
  - 3.1. Description of Building Blocks
  - 3.2. Verifier Agent Management
    - 3.2.1. Description
    - 3.2.2. Actors
  - 3.3. Usage of Generic Verifier
- 4. Detailed Class View
  - 4.1. OID4VP Presentation Verification
    - 4.1.1. Main Classes
    - 4.1.2. Class Diagram
      - 4.1.2.1. PresentationVerificationUsecase (Deprecated)
      - 4.1.2.2. DcqlVpTokenVerifier
- 5. Interfaces
- 6. Interface - Verifier Management API Specification (IF-100)
  - 6.1. Overview
  - 6.2. Data Model Summary
  - 6.3. Endpoints
    - 6.3.1. Create Verification
      - 6.3.1.1. Purpose
      - 6.3.1.2. Request Body Schema (CreateVerificationManagementDto)
      - 6.3.1.3. Example Request with dcql
    - 6.3.2. Get Verification
      - 6.3.2.1. Purpose
      - 6.3.2.2. Successful Response (200)
      - 6.3.2.3. Error Responses
  - 6.4. Field & Enum Reference
    - 6.4.1. ResponseModeTypeDto
    - 6.4.2. VerificationStatusDto
    - 6.4.3. VerificationErrorResponseCodeDto (subset)
    - 6.4.4. DCQL Note
  - 6.5. Examples
    - 6.5.1. Create Verification (cURL)
    - 6.5.2. Poll Verification
    - 6.5.3. Handling Not Found
- 7. Interface - Verification Controller API Specification (IF-101)
  - 7.1. Overview
  - 7.2. Headers
  - 7.3. Data Model Summary
  - 7.4. Endpoints
    - 7.4.1. Get OpenID Client Metadata
    - 7.4.2. Get Request Object
    - 7.4.3. Submit Verification Presentation
      - 7.4.3.1. Example Request with dcql
      - 7.4.3.2. Example payload encrypted with DCQL (Only available on SWIYU-API-Version: "2")
  - 7.5. Error Handling
  - 7.6. Field & Enum Reference
    - 7.6.1. VPApiVersion
    - 7.6.2. VerificationPresentationUnionDto (form fields)
    - 7.6.3. RequestObjectDto (JSON or JWT content)
    - 7.6.4. OpenidClientMetadataDto
  - 7.7. Examples
    - 7.7.1. Fetch Client Metadata
    - 7.7.2. Retrieve Request Object (JSON or JWT)
    - 7.7.3. Submit Rejection
    - 7.7.4. Submit V1/DCQL Presentation
    - 7.7.5. Submit Encrypted DCQL (JWE) (SWIYU-API-Version: 2)
- 8. Runntime - Verification Process - Complete
  - 8.1. Verification Process - Detailed
    - 8.1.1. 1. Verification Request Creation
    - 8.1.2. 2. Forwarding to the Holder
    - 8.1.3. 3. Holder Retrieves the Request
    - 8.1.4. 4. Optional Signing
    - 8.1.5. 5. Holder Sends Verifiable Presentation
    - 8.1.6. 6. Verification of Presentation
    - 8.1.7. 7. Checking Verification Status (Loop)
- 9. Runntime - Verification Process - Holder Rejection

- 10. Runntime - Trust Infrastructure Interactions
- 11. Runntime - Get Status list - Detailed
- 12. Runntime - Get Public Key - Detailed
- 13. Runntime - Implementing DCQL in Generic Verifier Service
    - 13.1.1. 1. Verification Request Creation
    - 13.1.2. 2. Forwarding to the Holder
    - 13.1.3. 3. Holder Retrieves the Request
    - 13.1.4. 4. Optional Signing
    - 13.1.5. 5. Holder Sends Verifiable Presentation
    - 13.1.6. 6. Verification of Presentation
    - 13.1.7. 7. Checking Verification Status (Loop)
  - 13.2. Changes in Interface for switching from DIF Presentation Exchange (DIF PE) to Digital Credentials Query Language (DCQL)
    - 13.2.1. 1. /management/api/verifications (POST) – Create a Verification Request
    - 13.2.2. 2. /management/api/verifications/{verificationId} (GET) – Retrieve Verification Status
    - 13.2.3. 3. /oid4vp/api/request-object/{request_id} (GET) – Retrieve Request Object by Holder
    - 13.2.4. 4. /oid4vp/api/request-object/{request_id}/response-data (POST) – Receive Verifiable Presentation
    - 13.2.5. DCQL Structure (DcqlQueryDto) in components/schemas
      - 13.2.5.1. Nested Schema: DcqlCredentialDto (Credential Query)
      - 13.2.5.2. Nested Schema: DcqlCredentialMetaDto
      - 13.2.5.3. Nested Schema: DcqlClaimDto (Claims Query)
  - 13.3. Example
- 14. Runntime - Validate Issuer via Trust Registry (TP2 Verifier-View)
  - 14.1. Sequence Diagram Verifier-View
  - 14.2. Caching for TP2 Verifier-View
  - 14.3. Evaluate TP2 Trust Markers
    - 14.3.1. Phase 1: Base Validation of Trust Statements
    - 14.3.2. Phase 2: Specific Verification of Trust Markers
- 15. Runntime - Self-Service Flow for Verifiers to Register vqPS
  - 15.1. On-the-Fly Registration Flow
- 16. Runntime - Sidechannel TS in the Generic Verifier
- 17. Crosscutting - Branches / Releases Concept
  - 17.1. Naming Conventions of Branches
  - 17.2. Semantic Versioning (SemVer) Overview
    - 17.2.1. Security Fix Process on release branch
  - 17.3. Releases
    - 17.3.1. Promotion Workflow
    - 17.3.2. When to create a official release

1. Business Context

1.1. Use Cases



UC Number	Name	Description	Link
UCV_E1	Display QR Code for Verification	The verifier displays a QR code containing a Request URI. This QR code acts as a deeplink to the verification request details, allowing the holder to initiate the verification process by scanning it.	

UCV_E2	Scan QR Code and Fetch Deeplink	The holder scans the QR code presented by the verifier. The wallet fetches the deeplink, which contains the information needed to proceed with the verification process.	
UCV_O1	Request Verification Object	The wallet fetches the openid connect request object from the verifier service to obtain a the business verifier's query and response transmission information.	
UCV_O1a	Create Signed Request Object	The verifier service creates a signed request object, ensuring the integrity and authenticity of the verification request data.	
UCV_O1b	Create Unsigned Request Object	The verifier service creates an unsigned request object, which may be used in scenarios where signing is not required or for testing purposes.	
UCV_O1c (new)	Embed Trust Statements in verifier_info	Retrieves the active Verification Query Public Statement (vqPS) from the database and embeds it as a JWT into the `verifier_info` claim of the Authorization Request. This provides the Wallet with the required cryptographic proof of the verification's declared purpose before any data is shared.	
UCV_O2	Submit Verification Presentation	The wallet submits a verification presentation to the verifier service. This presentation contains the credentials and proofs required for verification.	
UCV_O2a	Verify Presentation Submission and SD-JWT Credential	The verifier service checks the submitted presentation and validates the SD-JWT credential to ensure it meets the verification requirements.	
UCV_O2c	Check VC Status in Registry	The verifier service checks the status of the Verifiable Credential (VC) in a registry to confirm its validity, revocation status, or other relevant attributes.	
UCV_O2d	Store Verification Result	The verifier service stores the result of the verification process to DB	
UCV_O2f (new)	Validate Issuer via Trust Registry (TP2 Verifier-View)	As part of the token verification (`UCV_O2a`), the Generic Verifier contacts the Trust Registry to validate the identity and authorization of the VC issuer. For this purpose, the Trust Statements (such as `idTS`, `ncTLS`, `piTLS`, `piaTS`) are retrieved and the TP2 Trust Markers are evaluated	6.2.7 Validate Issuer via Trust Registry (TP2 Verifier-View)
UCV_O2e	Trigger Webhook Callback	After verification, the verifier service triggers a webhook callback to notify the business verifier system that a result is ready.	4.3. Webhook Communication Generic Components with Business Components
UCV_M1	Initiate Verification Process	The business verifier initiates a new verification process, starting the workflow for a new verification request defining the data they wish to know of the holder.	
UCV_M1a	Create Verification Management Entry	A new entry is created in the verification management system to track the verification process and its state.	
UCV_M1b	Provide Deeplink for QR Code	The verification management system provides a deeplink that can be embedded in a QR code, linking to the verification request.	
UCV_M1c (new)	Register vqPS on-the-fly	Automatically registers an unknown or expiring DCQL query with the Trust Management System (TMS) during the creation of a verification request. It fetches and persists the resulting signed Verification Query Public Statement (vqPS) along with its scope to ensure public transparency.	
UCV_M3	Retrieve Verification Result	The business verifier retrieves the result of the verification process, especially in cases where a webhook callback is not used.	
UCV_M4 (new)	Retrieve Full Verifiable Presentation (Extended Non-Repudiation)	Allows the Business Verifier to retrieve the entire Verifiable Presentation (VP) on demand, instead of just the extracted VC attributes. This primarily serves use cases (such as AGOV) that require extended, cryptographically provable non-repudiation for later audits.	

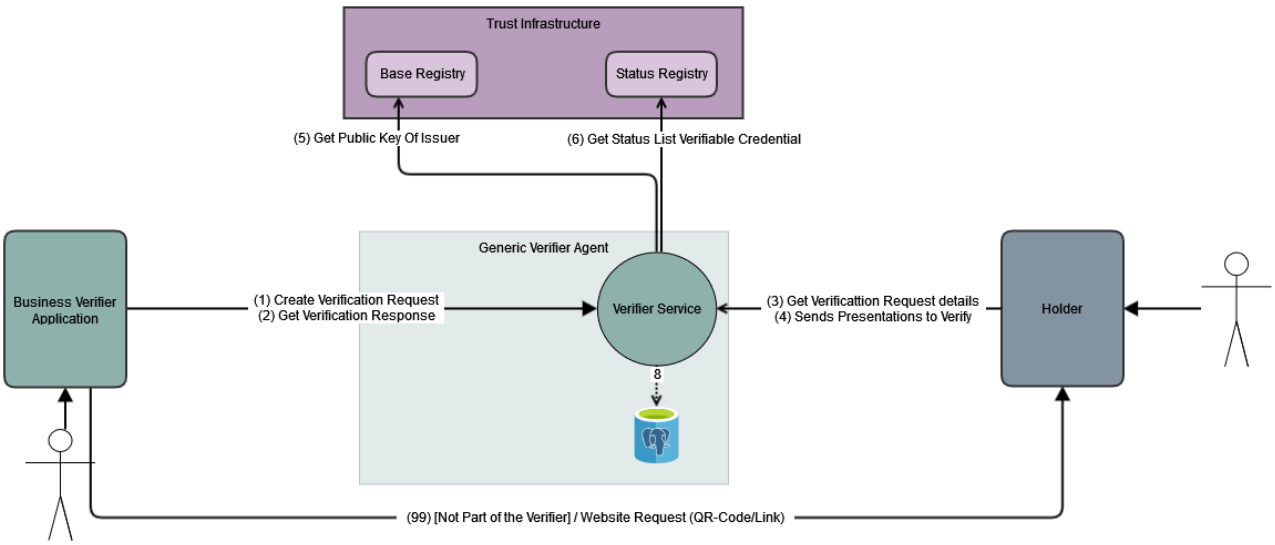
1.2. Actors

Actor	Description
Business Verifier Application <<system>>	Application that initializes the verification process and requests the verification result.
Holder Wallet <<system>>	Mobile Wallet for iOS and Android which holds and builds the verifiable presentations.
Holder	Person which has a VC on his Wallet
Verifier	Person or System which would like to get and validate your VC

1.3. External Systems

System	Description
Base Registry	Service to store public available data used for the verification process (for example to get the public key for the issuer). The service is part of the Trust Infrastrcture.
Status Registry	Is part of the Trust Infrastructure. Stores the Status Lists created by the Generic Issuer.The service is part of the Trust Infrastrcture.

1.4. Interactions



Number	Interaction origin	Interaction Target	Description
1	Business Verifier	Verifier Service (Management)	Business Verifier creates a verification request with an Api-call. As a response it receives a verification-management dto including the VerificationStatus and verificationUrl.
2	Business Verifier	Verifier Service (Management)	Business Verifier requests the verification request result with an Api-call. As a response it receives a verification-management dto including the VerificationStatus, WalletResponse and verificationUrl.
3	Holder (Wallet)	Generic Verifier OID4VP Service	Holder requests the verification details, received from the Business Verifier (99), from the validator via Api-call.
4	Holder (Wallet)	Generic Verifier OID4VP Service	Holder sends the requested verification details or rejects the verification to the validator via Api-call.
5	Verifier Service (OID4VP)	Trust Infrastructure - Base Registry	Validator gets the public key from the Issuer, which is part of the sent verifiable presentation as a did:webvh, via Api-call.
6	Verifier Service (OID4VP)	Trust Infrastructure - Status Registry	Validator gets the Status of the presentation from the Status Registry via Api-call.

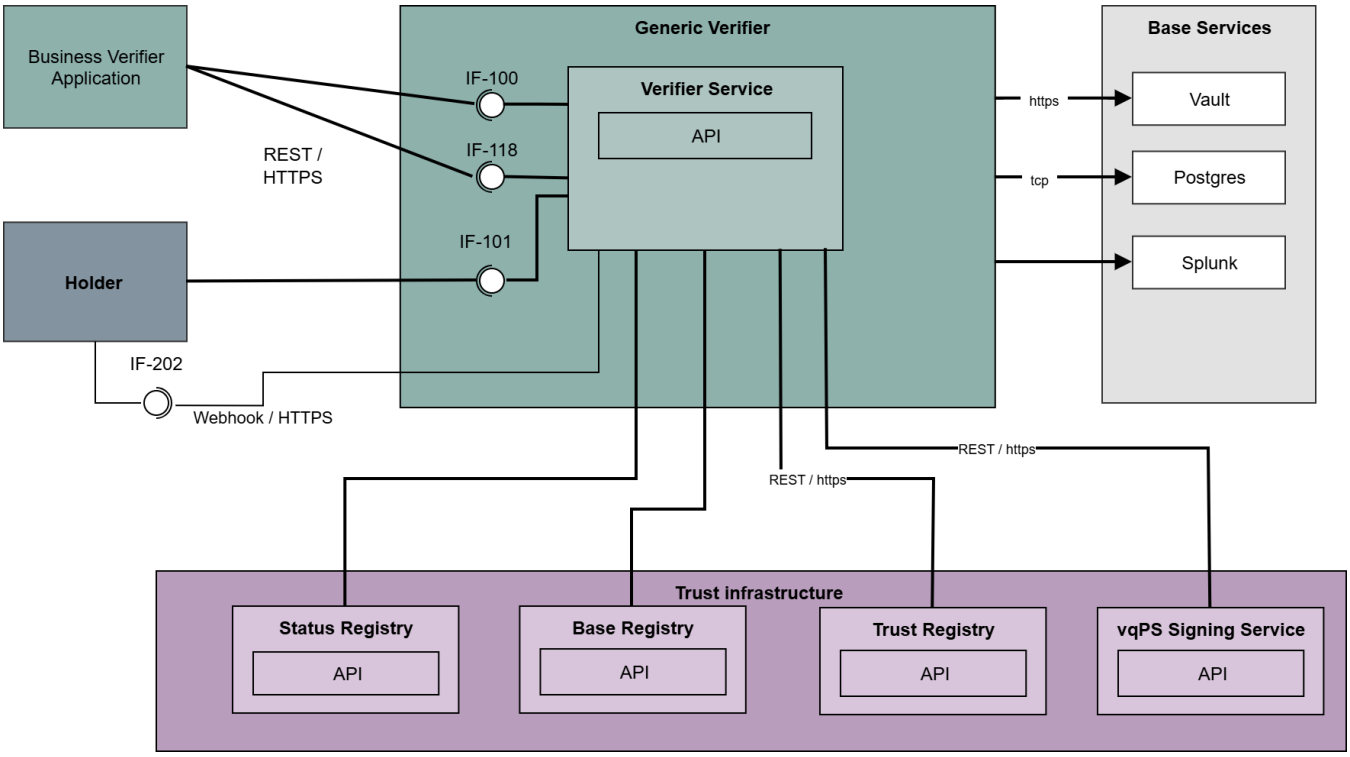
There are no direct interactions between Verifier Management Service and Verifier Validator Service. All interactions are exclusively via Database.

1.5. Additional Information

Business Verifier Application communicates exclusively with the Generic Verifier Management Service. The Holder Communicates only with the validator.

2. Technical Context

The technical context shows how the Generic Verifier is embedded in it's environment and shows its interactions with it.



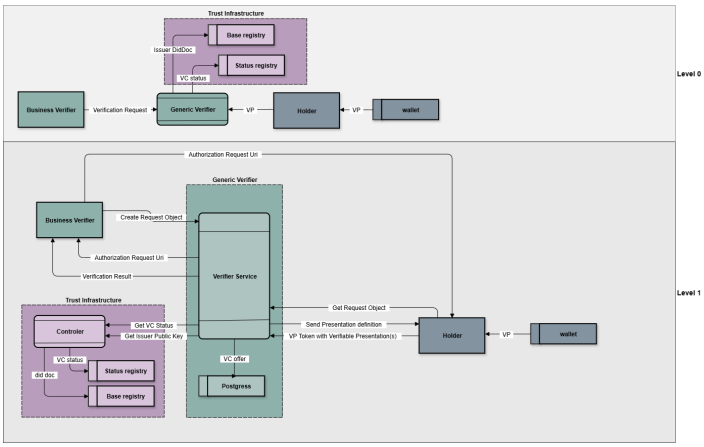
2.1. Core System

System	Description
Verifier-Management	Component to start and check the status of the verification process. The component provides an API for the "Business Verifier Application". The holder is not interacting with this service.
Verifier-Validator	Component to handle the actual verification process. It interacts only with the holder via API.

2.2. Context

System	Description
Postgres	Database to store the business objects.
Splunk	Used for application logging and audit logs.
Vault	Secure storage of credentials.
HSM	Hardware Security Module used to generate/store keys in a secure maner.

3. Building Block View



3.1. Description of Building Blocks

Name	Responsibility
Verifier Service	The Verifier Service is the API used by the Business Verifier Application. It is used to initialize the Verification Process and to return the Status of a request upon request. It creates the verification entry in the Database. It checks the validity of the VC with data from the Status Registry and the integrity of the Presentation with the the base registry.

Refer to the Issuer Service page for the description of the other actors and building blocks.

3.2. Verifier Agent Management

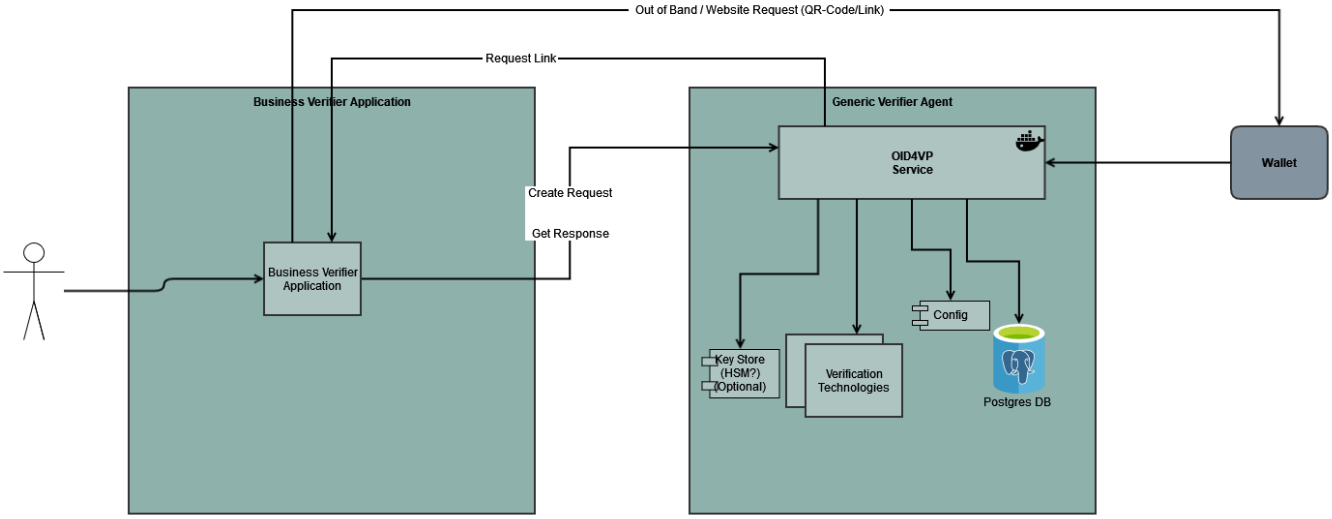
3.2.1. Description

The Verifier Agent Management Service is the API used by the Business Verifier Application. It is used to initialize the Verification Process and to return the Status of a request upon request. It creates a the verification in the Database, which is later used by the OID4VP Service.

3.2.2. Actors

Actor	Scope	Entity	Actions
Business Verifier Application	Internal		• Creates Verification Process • Requests Verification Status • Passes link for the actual verification mechanism to Holder Application.

3.3. Usage of Generic Verifier



4. Detailed Class View

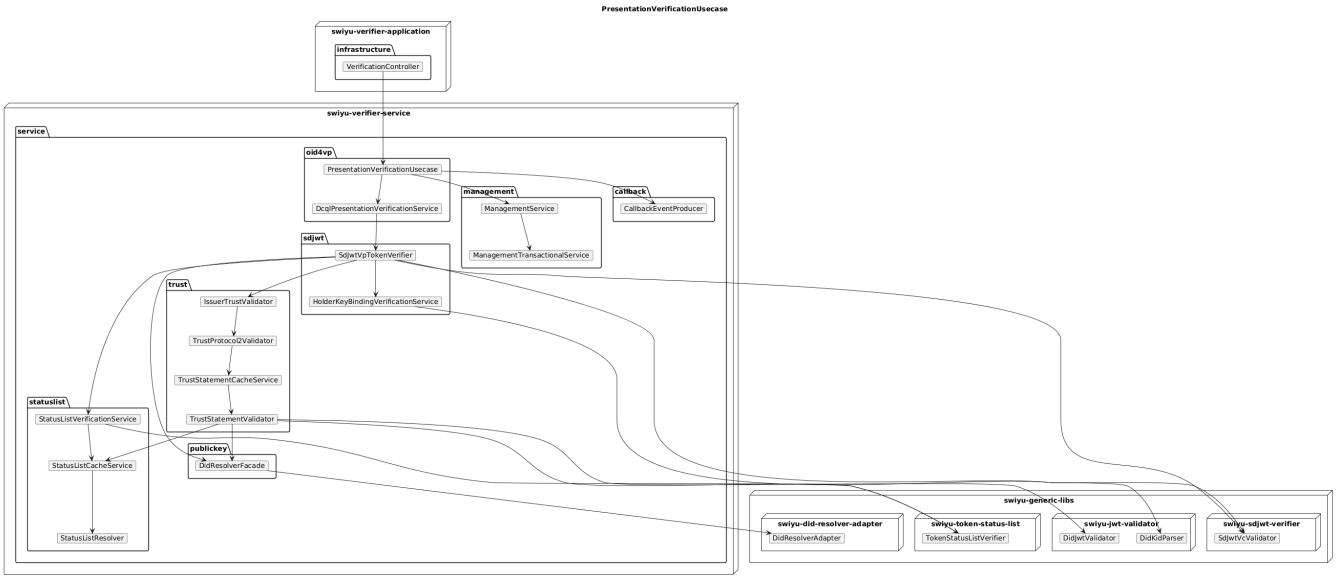
4.1. OID4VP Presentation Verification

This package contains the application logic for the verification of presentations according to OID4VP – including a dedicated DCQL flow.

4.1.1. Main Classes

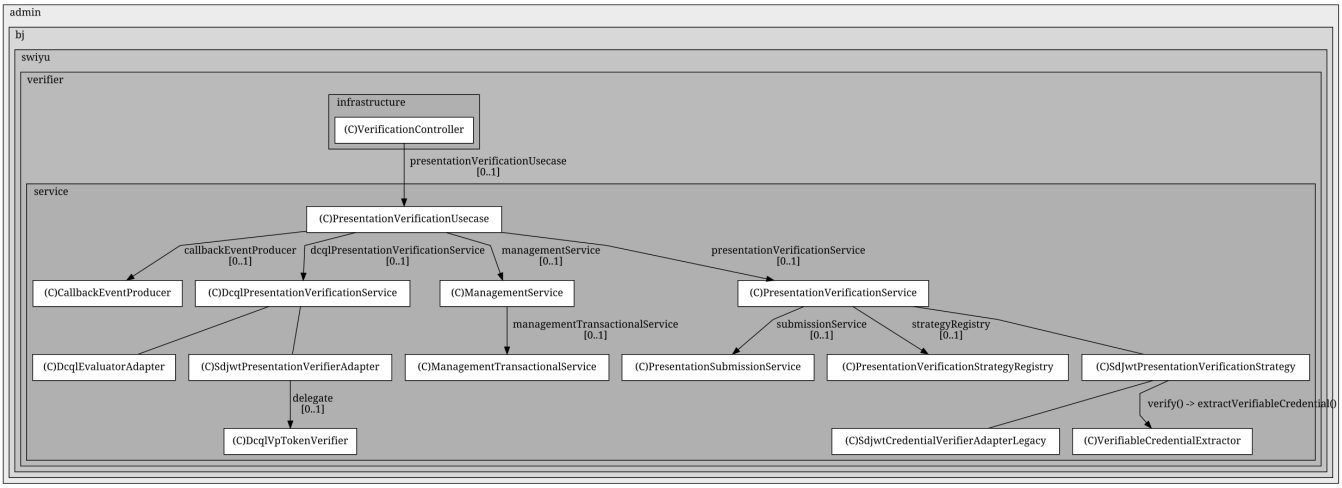
- `PresentationVerificationUsecase`
  - Central orchestration of the verification.
  - Loads the `Management` entity, checks the status, delegates to services and writes the result (success/fail).
  - Always triggers a callback event via `CallbackEventProducer` at the end.
- `PresentationVerificationService`
  - Service for the **standard OID4VP flow**.
  - Parses and validates `presentation_submission` via `PresentationSubmissionService`.
  - Determines the format from the `descriptor_map` and selects the appropriate `PresentationVerificationStrategy` via the `PresentationVerificationStrategyRegistry`.
- `DcqlPresentationVerificationService`
  - Service for the **DCQL flow**.
  - Uses the DCQL query stored in the `Management` entity, verifies the provided VP tokens via `PresentationVerifier<SdJwt>` (typically `VpTokenVerifierAdapter`) and checks DCQL claims via `DcqlEvaluator`.
- `PresentationVerificationStrategyRegistry`
  - Registry for `PresentationVerificationStrategy` implementations.
  - Maps `format` (e.g. `"vc+sd-jwt"`, `"dc+sd-jwt"`) to the corresponding strategy.
- `VpTokenVerifier`
  - Domain service for the verification of SD-JWT / SD-JWT VC:

4.1.2. Class Diagram

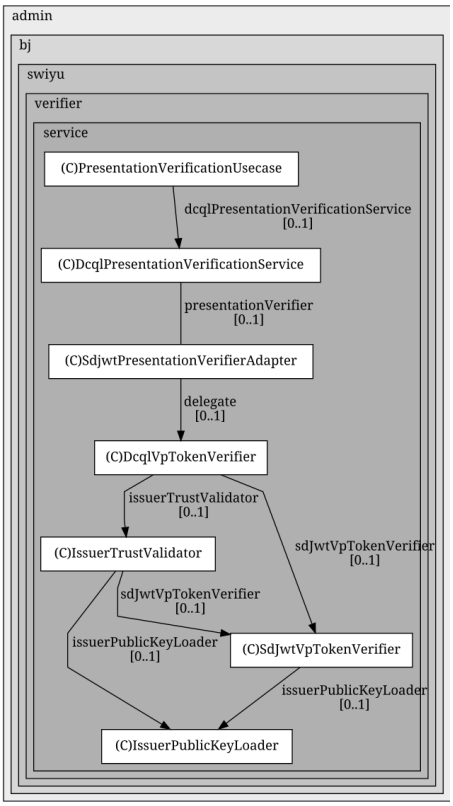


(Diagram as plantuml is in code, because of old plantuml in confluence)

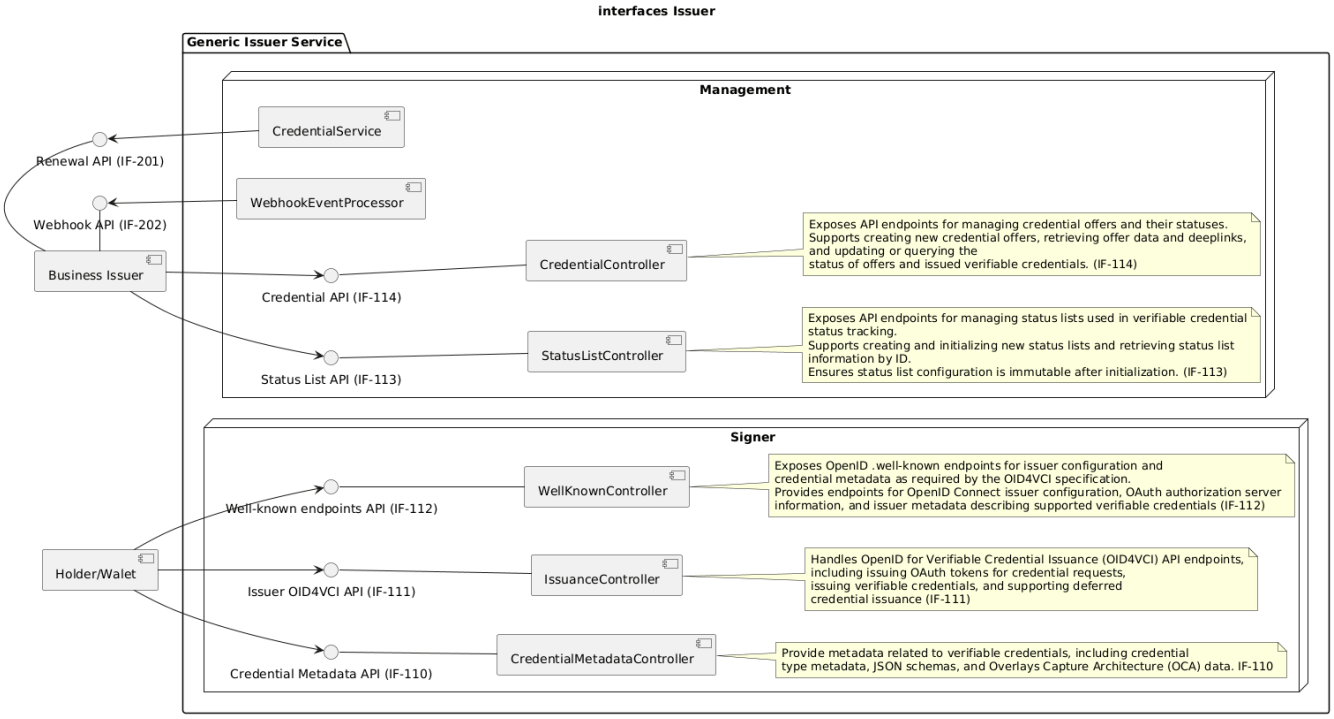
4.1.2.1. PresentationVerificationUsecase (Deprecated)



4.1.2.2. DcqlVpTokenVerifier



5. Interfaces



6. Interface - Verifier Management API Specification (IF-100)

This document describes the Management endpoints exposed by `VerifierManagementController` under the base path:

`/management/api/verifications`

The API allows a **Business Verifier to (1) create a new verification process and (2) retrieve the current status and data of an existing verification process**. It follows concepts from OpenID for Verifiable Presentations (OID4VP) and supports DCQL queries.

6.1. Overview

The Management API lets a Verifier initiates a verification request specifying accepted issuers, trust anchors, DCQL credential queries and configuration overrides. Once created, a verification object can be polled to observe state transitions (`PENDING` `SUCCESS` | `FAILED`) and retrieve wallet response data (credential subject claims or error context).

6.2. Data Model Summary

Primary DTOs (request/response):

- `CreateVerificationManagementDto` – Request body for creating a verification.
- `ManagementResponseDto` – Response body for both creation and retrieval. Supporting DTOs & Enums:
- `DcqlQueryDto` (Digital Credentials Query Language) – currently marked as “not implemented” but accepted structurally.
- `ConfigurationOverrideDto` – Per-request overrides of verifier configuration.
- `ResponseModeTypeDto` – Expected wallet response mode (`direct_post` or `direct_post.jwt`).
- `VerificationStatusDto` – State machine values: `PENDING`, `SUCCESS`, `FAILED`.
- `ResponseDataDto` – Wallet response containing the business error code, error description and credential subject data.
- `VerificationErrorResponseCodeDto` – Detailed error code enum.
- `ApiErrorDto` – Error payload for HTTP errors (e.g. 404 Not Found).

6.3. Endpoints

6.3.1. Create Verification

Create a new verification process identified by a server-generated UUID.

```
POST /management/api/verifications
Content-Type: application/json
```

6.3.1.1. Purpose

Initiates a verification flow. The returned object includes the unique identifier and URLs/deeplinks the wallet can use to conduct the presentation.

6.3.1.2. Request Body Schema (`CreateVerificationManagementDto`)

Field	Type	Required	Description
accepted_issuer_dids	array[string]	Conditional (see validation)	Explicit allow-list of issuer DIDs. Evaluated before trust protocol.
jwt_secured_authorization_request	boolean	Optional	If true, the authorization request object is JWT-secured (signed by the verifier).
response_mode	`ResponseModeTypeDto`	Optional (default: `direct_post`)	Wallet response mode. `direct_post` = clear text; `direct_post.jwt` = encrypted/signed response.
configuration_override	`ConfigurationOverrideDto`	Optional	Per-request overrides (external URL, DID, key selection, etc.).
dcql_query	`DcqlQueryDto`	Optional	DCQL query (not fully implemented claim-sets and credential_sets are missing).
verification_purpose	`VerificationPurposeDto`	Optional	Transparency metadata for registration at the TMS. When present, the verifier registers the DCQL query and injects the resulting vqPS into subsequent Authorization Requests.
redirect_uri	`URI`	Optional	Absolute URI to redirect the user after the verification process. Must contain a session_nonce parameter.

Validation rule (`@AcceptedIssuerDidsOrTrustAnchorsNotEmpty`): At least one of `accepted_issuer_dids` or `trust_anchors` must be non-empty (unless business logic allows open trust; clarify if enforced at runtime).

6.3.1.3. Example Request with dcql

```
{
  "accepted_issuer_dids" : [ "did:webvh:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX:my-did:api:v1:did:
xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx" ],
  "jwt_secured_authorization_request" : true,
  "response_mode" : "direct_post",
  "configuration_override" : null,
  "redirect_uri": "https://example.com/redirect?session_nonce=12345",
  "verification_purpose": {
      "scope": "com.example.university-degree-verification",
      "purpose_name": {
          "default": "University Degree Verification",
          "de-ch": "Universitätsabschlussverifikation"
      },
      "purpose_description": {
          "default": "Verification of university degree"
      }
  },
  "dcql_query" : {
    "credentials" : [ {
      "id" : "VerifiableCredential",
      "format" : "vc+sd-jwt",
      "multiple" : false,
      "meta" : {
        "type_values" : [ [ "string" ], [ "string" ], [ "number" ] ],
        "vct_values" : [ "http://default-issuer-url.admin.ch/oid4vci/vct/my-vct-v01" ],
        "doctype_value" : null
      },
      "claims" : [ {
        "id" : null,
        "path" : [ "type" ],
        "values" : [ "Bachelor of Science" ]
      }, {
        "id" : null,
        "path" : [ "name" ],
        "values" : null
      }, {
        "id" : null,
        "path" : [ "average_grade" ],
        "values" : null
      } ],
      "claim_sets" : null,
      "require_cryptographic_holder_binding" : true,
      "trusted_authorities" : [ ]
    } ],
    "credential_sets" : [ ]
  }
}
```

`Successful Response (200)`

Body: `ManagementResponseDto`

Field	Type	Description
id	UUID	Unique identifier of verification process.
request_nonce	string	Nonce correlated with authorization request; used for replay protection.
state	`VerificationStatusDto`	Current status (`PENDING` on creation).
dcql_query	`DcqlQueryDto`	Echo of DCQL query (if provided).
credential_evaluation	`Map<String, List<CredentialEvaluationDto>>`	Object of Evaluations, where the key is the id of the dcql_query credential.
wallet_response	`ResponseDataDto`	Populated only after completion / error; null while pending.
verification_url	string	URL for initiating verification (wallet fetches request object).
verification_deeplink	string	Deep link (e.g., mobile scheme) to trigger wallet app directly.

Example Response:


```
{
  "id" : "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "request_nonce" : "uLCMnEhWqHyERMsjOOilNEh5tEwnNPfm",
  "state" : "PENDING",
  "dcql_query" : { ... },
  "wallet_response" : null,
  "verification_url" : "http://default-verifier-url.admin.ch/oid4vp/api/request-object/xxxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxxx",
  "verification_deeplink" : "swiyu-verify://?client_id=did:webvh:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX:
my-did:api:v1:did:xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx&request_uri=http//default-verifier-url.admin.ch/oid4vp
/api/request-object/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
}
```


Error Responses

Status	Reason	Body
400	Validation error (e.g., both `accepted_issuer_dids` and `trust_anchors` empty when required)	`ApiErrorDto`
500	Unexpected server error	`ApiErrorDto`

6.3.2. Get Verification

Retrieve the current state, response data, and parameters of a verification by its identifier.

`GET /management/api/verifications/{verificationId}`

Path Parameter:

Name	Type	Required	Description
verificationId	UUID	Yes	Identifier returned by creation endpoint.

6.3.2.1. Purpose

Allows polling or fetching the final outcome (`SUCCESS` or `FAILED`). Once `wallet_response` is available it contains credential subject data or error diagnostics.

6.3.2.2. Successful Response (200)

Body identical to creation response schema (`ManagementResponseDto`).

Example Response (`SUCCESS`):


```
{
  "id" : "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "request_nonce" : "bisgDTA2D0f8CphGiakzIcajjwkw+JvT",
  "state" : "SUCCESS",
  "dcql_query" : {
        "credentials": [
                "id": "VerifiableCredential",
                "format": "dc+sd-jwt",
                "meta": {
                        "vct_values": ["http://default-issuer-url.admin.ch/oid4vci/vct/my-vct-v01"]
                },
                "claims": [{
                        "path": ["average_grade"]
```

```
                }]
        ]
  },
  "wallet_response" : {
    "error_code" : null,
    "error_description" : null,
    "dcql_response": {
      "VerifiableCredential": {
         "average_grade" : ["5.33"]
      }
    },
    "credential_subject_data" : {
      "VerifiableCredential" : [ {
        "_sd" : [ "Hm7v-Ws47cVlXO3VCLQemNE9MWc4kDUT_7qXX3bq7l8", "bmD7qA9d3aWvwPeiNTXofKVKXeEnMpMk7rHPzfUtDsM",
"u4SWDHoCaEbY0c8qmSSY7dNnp5gNefICLuyqEyJqPFw" ],
        "vct" : "http://default-issuer-url.admin.ch/oid4vci/vct/my-vct-v01",
        "_sd_alg" : "sha-256",
        "iss" : "did:webvh:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX:my-did:api:v1:did:xxxxxxxx-xxxx-xxxx-
xxxx-xxxxxxxxxxxx",
        "cnf" : {
          "kty" : "EC",
          "crv" : "P-256",
          "kid" : "wallet-proof-key-1",
          "x" : "rQX5t_ICGBEgxLg3YBgbCoGZUDstIUGqy4wbD5Z8l4E",
          "y" : "fEarXBwgoFq69KpUctSpjpd7YJaggl1tpt1hKenAXGA",
          "jwk" : {
            "kty" : "EC",
            "crv" : "P-256",
            "kid" : "wallet-proof-key-1",
            "x" : "rQX5t_ICGBEgxLg3YBgbCoGZUDstIUGqy4wbD5Z8l4E",
            "y" : "fEarXBwgoFq69KpUctSpjpd7YJaggl1tpt1hKenAXGA"
          }
        },
        "iat" : "2025-12-02T13:54:32.000+00:00",
        "status" : {
          "status_list" : {
            "type" : "SwissTokenStatusList-1.0",
            "idx" : 7166,
            "uri" : "https://my-did/api/v1/statuslist/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx.jwt"
          }
        },
        "average_grade" : "5.33",
        "name" : "Data Science",
        "type" : "Bachelor of Science"
      } ]
    },
    "vp_token": {
                "VerifiableCredential": ["ey....."]
    }
  },
  "credential_evaluation": {
     "VerifiableCredential": [{
                  "credential_status": {
                         "valid": true,
                        "status" 0
                }
                "trust_markers": {
            "trust_method": "TRUST_PROTOCOL_2_0"
                        "is_trusted": true,
                        "viTM": true,
                        "caTM": true,
                        "gucTM": true,
                        "gucaTM": true
                }]
        }]
  }
  "verification_url" : "http://default-verifier-url.admin.ch/oid4vp/api/request-object/xxxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxxx",
  "verification_deeplink" : "swiyu-verify://?client_id=did:webvh:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX:
my-did:api:v1:did:xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx&request_uri=http://default-verifier-url.admin.ch/oid4vp
/api/request-object/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
```

```
}
```

Example Response (`FAILED`) due to invalid submission (unreliable data):

```
{
  "id" : "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "request_nonce" : "xmZ3d/NbhJt/XHL/dGeju+5Uj0qxTAOa",
  "state" : "FAILED",
  "dcql_query" : { ... },
  "wallet_response" : {
    "error_code" : "invalid_presentation_submission",
    "error_description" : "Not all requested claim values are satisfied",
    "credential_subject_data" : null
  },
  "verification_url" : "http://default-verifier-url.admin.ch/oid4vp/api/request-object/xxxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxxx",
  "verification_deeplink" : "swiyu-verify://?client_id=did:webvh:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX:
my-did:api:v1:did:xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx&request_uri=http://default-verifier-url.admin.ch/oid4vp
/api/request-object/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
}
```

Example Response (`FAILED`) due to incorrect VC state:

Note; This is the same as a successful response, only the state is failed.

```
{
  "id" : "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "request_nonce" : "bisgDTA2D0f8CphGiakzIcajjwkw+JvT",
  "state" : "FAILED",
  "dcql_query" : {
        "credentials": [
                "id": "VerifiableCredential",
                "format": "dc+sd-jwt",
                "meta": {
                        "vct_values": ["http://default-issuer-url.admin.ch/oid4vci/vct/my-vct-v01"]
                },
                "claims": [{
                        "path": ["average_grade"]
                }]
        ]
  },
  "wallet_response" : {
    "error_code" : null,
    "error_description" : null,
    "dcql_response": {
      "VerifiableCredential": {
         "average_grade" : ["5.33"]
    },
    "credential_subject_data" : {
      "VerifiableCredential" : [ {
        "_sd" : [ "Hm7v-Ws47cVlXO3VCLQemNE9MWc4kDUT_7qXX3bq7l8", "bmD7qA9d3aWvwPeiNTXofKVKXeEnMpMk7rHPzfUtDsM",
"u4SWDHoCaEbY0c8qmSSY7dNnp5gNefICLuyqEyJqPFw" ],
        "vct" : "http://default-issuer-url.admin.ch/oid4vci/vct/my-vct-v01",
        "_sd_alg" : "sha-256",
        "iss" : "did:webvh:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX:my-did:api:v1:did:xxxxxxxx-xxxx-xxxx-
xxxx-xxxxxxxxxxxx",
        "cnf" : {
          "kty" : "EC",
          "crv" : "P-256",
          "kid" : "wallet-proof-key-1",
          "x" : "rQX5t_ICGBEgxLg3YBgbCoGZUDstIUGqy4wbD5Z8l4E",
          "y" : "fEarXBwgoFq69KpUctSpjpd7YJaggl1tpt1hKenAXGA",
          "jwk" : {
            "kty" : "EC",
            "crv" : "P-256",
```

```
            "kid" : "wallet-proof-key-1",
            "x" : "rQX5t_ICGBEgxLg3YBgbCoGZUDstIUGqy4wbD5Z8l4E",
            "y" : "fEarXBwgoFq69KpUctSpjpd7YJaggl1tpt1hKenAXGA"
          }
        },
        "iat" : "2025-12-02T13:54:32.000+00:00",
        "status" : {
          "status_list" : {
            "type" : "SwissTokenStatusList-1.0",
            "idx" : 7166,
            "uri" : "https://my-did/api/v1/statuslist/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx.jwt"
          }
        },
        "average_grade" : "5.33",
        "name" : "Data Science",
        "type" : "Bachelor of Science"
      } ]
    },
    "vp_token": {
                "VerifiableCredential": ["ey....."]
    }
  },
  "credential_evaluation": {
     "VerifiableCredential": [{
                  "credential_status": {
                         "valid": false,
                        "status": 2
                }
                "trust_markers": {
            "trust_method": "TRUST_PROTOCOL_2_0"
                        "is_trusted": true,
                        "viTM": true,
                        "caTM": true,
                        "gucTM": true,
                        "gucaTM": true
                }]
        }]
  }
  "verification_url" : "http://default-verifier-url.admin.ch/oid4vp/api/request-object/xxxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxxx",
  "verification_deeplink" : "swiyu-verify://?client_id=did:webvh:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX:
my-did:api:v1:did:xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx&request_uri=http://default-verifier-url.admin.ch/oid4vp
/api/request-object/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
}
```


6.3.2.3. Error Responses

Status	Reason	Body
404	Verification not found or expired	`ApiErrorDto` (message from `VerificationNotFoundException`)
500	Unexpected server error	`ApiErrorDto`

Example 404 Body:


```
{
  "error": null,
  "error_description": null,
  "detail": "The verification with the identifier '2f7b5c89-8c29-4f31-9d9d-7c6c0a3e6b21' was not found"
}
```

6.4. Field & Enum Reference

6.4.1. ResponseModeTypeDto

Value	Meaning
direct_post	Wallet sends clear text response.
direct_post.jwt	Wallet sends JWT-secured response (encrypted/signed).

6.4.2. VerificationStatusDto

Value	Meaning
PENDING	Awaiting wallet presentation.
SUCCESS	Presentation validated; credential subject data available.
FAILED	Presentation failed; see `wallet_response.error_code`.

6.4.3. VerificationErrorResponseCodeDto (subset)

Code	Description
credential_invalid	Credential deemed invalid (generic).
jwt_expired	Expired JWT encountered.
jwt_premature	JWT not yet valid.
missing_nonce	Nonce absent where required.
credential_revoked	Credential revoked in status list.
credential_suspended	Credential suspended in status list.
holder_binding_mismatch	Holder binding cryptographic proof mismatch.
issuer_not_accepted	Issuer not in allow-list / trust chain.
malformed_credential	Credential does not meet format spec.
authorization_request_object_not_found	Request object unavailable.
verification_process_closed	Process already closed.
invalid_presentation_definition	Definition malformed or violated.
presentation_submission_constraint_violated	Constraints not satisfied.
vp_formats_not_supported	Wallet lacks requested format support.
invalid_presentation_definition_uri	URI unreachable.
invalid_presentation_definition_reference	Definition missing at reachable URI.

(See enum for full list.)

6.4.4. DCQL Note

`DcqlQueryDto` may contain fields with implementation status `not-implemented or experimental`.

6.5. Examples

6.5.1. Create Verification (cURL)

```
curl -X POST https://verifier.example.com/management/api/verifications \
  -H 'Content-Type: application/json' \
  -d '{
    "accepted_issuer_dids": [
    "<Your ISSUER_DID>"
  ],
  "jwt_secured_authorization_request": true,
  "response_mode": "direct_post",
  "verification_purpose": {
    "scope": "ch.some.test.scope",
    "purpose_name": {
      "default": "Test"
    },
    "purpose_description": {
      "default": "This is a test and this its description"
    }
  },
  "dcql_query": {
    "credentials": [
      {
        "id": "00000000-0000-0000-0000-000000000000",
        "format": "dc+sd-jwt",
        "meta": {
          "vct_values": ["betaid-sdjwt"]
        },
        "claims": [
          {
            "path": ["age_over_18"]
          }
        ],
        "require_cryptographic_holder_binding": true
      }
    ]
  }
  }'
```

6.5.2. Poll Verification

```
curl -X GET \
  https://verifier.example.com/management/api/verifications/2f7b5c89-8c29-4f31-9d9d-7c6c0a3e6b21
```

6.5.3. Handling Not Found

```
curl -i -X GET \
  https://verifier.example.com/management/api/verifications/00000000-0000-0000-0000-000000000000
```

Response:

```
HTTP/1.1 404 Not Found
Content-Type: application/json

{"detail":"The verification with the identifier '00000000-0000-0000-0000-000000000000' was not found"}
```

7. Interface - Verification Controller API Specification (IF-101)

This document describes the OpenID for Verifiable Presentations (OID4VP) endpoints exposed by `VerificationController` under the base path:

```
/oid4vp/api/
```

The API is intended for Wallets to interact with a Verifier by fetching OpenID Client Metadata, retrieving the Authorization Request Object (JSON or signed JWT), and submitting verification presentations (including rejections, Presentation Exchange, DCQL, and encrypted DCQL).

7.1. Overview

The `VerificationController` implements the OID4VP endpoints used by Wallets:

- Provide Verifier metadata for discovery and capability negotiation.
- Provide the Authorization Request Object to the Wallet, either as a signed JWT or as JSON.
- Accept the Wallet's verification response via form-urlencoded POST, with support for multiple modes and formats, including DCQL.

Tag: Verfifier OID4VP API (IF-101)

7.2. Headers

Notable Headers:

- SWIYU-API-Version (optional): distinguishes request/response format semantics.
  - "1" VPApiVersion.ID2 (DIF Presentation Exchange, OID4VP Draft, Implementers Draft 2)
  - "2" VPApiVersion.V1 (OID4VP 1.0 with DCQL)
  - If absent, defaults to ID2.

7.3. Data Model Summary

Primary DTOs:

- `OpenidClientMetadataDto` – Verifier's OpenID client metadata.
- `RequestObjectDto` – Authorization Request Object (JSON structure; may be sent as signed JWT instead).
- `VerificationPresentationUnionDto` – Union of all possible POST form fields for Wallet submission.

Supporting Models & Enums:

- `VPApiVersion` – API version header enum, values "1" (ID2) and "2" (V1).
- `ApiErrorDto` – Error payload for HTTP errors (e.g. 404 Not Found).
- `VerificationErrorResponseCode` – Error codes returned for more granular information when rejecting an incorrect verification. Causes are faults in wallet implementation such as the wrong vc is presented, not all request claims being disclosed, the key binding is adressed to another verifer.

7.4. Endpoints

7.4.1. Get OpenID Client Metadata

```
GET /oid4vp/api/openid-client-metadata.json
```

Purpose:

- Returns the Verifier's metadata so the Wallet can understand name, logo, supported formats, and crypto capabilities.

Response (200, application/json)

Body: `OpenidClientMetadataDto` Key fields:

- client_id: string (required)
- vp_formats_supported: object describing supported formats, e.g., `dc+sd-jwt` and algorithm lists
- jwks: JWK Set for key agreement/encryption (not for verifying signed Authorization Requests)
- encrypted_response_enc_values_supported: list of JWE enc algorithms (default includes "A128GCM")
- additionalProperties: other metadata values (via JsonAnySetter/Getter), e.g., localized names and logos

Example:

```
{
  "version": "1.0",
  "client_id": "did:example:12345",
  "vp_formats_supported": {
     "dc+sd-jwt": {
         "sd-jwt_alg_values": [
            "ES256",
            "Ed25519"
         ],
         "kb-jwt_alg_values": [
             "ES256",
             "Ed25519"
        ]
     }
  }
  "jwks": null,
  "encrypted_response_enc_values_supported": null,
  "client_name#de": "Entwicklungs-Demo-Verifizierer (Fallback DE)",
  "client_name#de-CH": "Entwickligs-Demo-Verifizier",
  "client_name#fr": "Vérificateur de démonstration de développement",
  "client_name#de-DE": "Entwicklungs-Demo-Verifizierer",
  "logo_uri": "www.example.com/logo.png",
  "client_name#en": "Development Demo Verifier",
  "logo_uri#fr": "www.example.com/logo_fr.png",
  "client_name": "DEV Demo Verifier (Base)"
}
```


7.4.2. Get Request Object

```
GET /oid4vp/api/request-object/{request_id}
Produces: application/json OR application/oauth-authz-req+jwt
```

Purpose:

- Provides the Authorization Request Object for the given verification request. Depending on verifier configuration (JAR flag), this is either:
  - A signed JWT (content type `application/oauth-authz-req+jwt`), or
  - A JSON `RequestObjectDto` (content type `application/json`).

Path parameters:

- request_id: UUID (required)

Responses:

- 200: Body is a string JWT or a `RequestObjectDto` JSON object.
- 404: `ApiErrorDto` if the request object is not found.

`RequestObjectDto` key fields:

- client_id: string (verifier DID or client identifier)
- response_type: string
- response_mode: `ResponseModeTypeDto` (e.g., `direct_post`, `direct_post.jwt`)
- response_uri: string (where the Wallet posts the response)
- nonce: string
- version: string
- dcql_query: `DcqlQueryDto` (for DCQL-based requests)
- client_metadata: `OpenidClientMetadataDto` (inline metadata; should not be present if client_metadata_uri is used externally)
- state: string

Example (JSON):

```
{
  "client_id" : "did:webvh:QmRsg8DGUEN34qdkvUSzbzjYLvJwBo5bwrFNVpnCy66bkz:mockserver%3A1080:api:v1:did:9f3a1d54-524b-4922-a263-83c0146e35ea",
  "client_id_scheme" : "did",
  "response_type" : "vp_token",
  "response_mode" : "direct_post.jwt",
  "response_uri" : "http://default-verifier-url.admin.ch/oid4vp/api/request-object/dfc7d288-ef9d-4b32-8c20-50aa6493098f/response-data",
  "nonce" : "uVvWO5JMulRHhgQa0s1wF6vKUlaFR984",
  "version" : "1.0",
  "dcql_query" : {
    "credentials" : [ {
      "id" : "VerifiableCredential",
      "format" : "vc+sd-jwt",
      "multiple" : false,
      "meta" : {
        "type_values" : [ [ "string" ], [ "string" ], [ "number" ] ],
        "vct_values" : [ "http://default-issuer-url.admin.ch/oid4vci/vct/my-vct-v01" ],
        "doctype_value" : null
      },
      "claims" : [ {
        "id" : null,
        "path" : [ "type" ],
        "values" : [ "Bachelor of Science" ]
      }, {
        "id" : null,
        "path" : [ "name" ],
        "values" : null
      }, {
        "id" : null,
        "path" : [ "average_grade" ],
        "values" : null
      } ],
      "claim_sets" : null,
      "require_cryptographic_holder_binding" : true,
      "trusted_authorities" : null
    } ],
    "credential_sets" : null
  },
  "client_metadata" : {
    "version" : "1.0",
    "additionalProperties" : { },
    "client_id" : "did:webvh:QmRsg8DGUEN34qdkvUSzbzjYLvJwBo5bwrFNVpnCy66bkz:mockserver%3A1080:api:v1:did:9f3a1d54-524b-4922-a263-83c0146e35ea",
    "jwks" : {
      "keys" : [ {
        "kty" : "EC",
        "kid" : "c869d127-d914-4ea1-8e9d-6bdc1cb86711",
        "use" : null,
        "alg" : null,
        "n" : null,
        "e" : null,
        "crv" : "P-256",
        "x" : "p3eHMtcs6UdLYRBoubUcZjtULBedxrw_ycvGOGsuFIs",
        "y" : "LCGFitciKQvydsffayyWvGENOdnShN7sZublJwUuspc"
      } ]
    },
    "encrypted_response_enc_values_supported" : [ "A128GCM" ],
    "vp_formats_supported" : null
  },
  "state" : null
}
```

Example (signed JWT):

```
eyJhbGciOiJFUzI1NiIsInR5cCI6IkpXVCJ9....
```

7.4.3. Submit Verification Presentation

```
POST /oid4vp/api/request-object/{request_id}/response-data
Consumes: application/x-www-form-urlencoded
Produces: application/json
```

Purpose:

- Receives the Wallet's verification response. The endpoint accepts multiple submission styles via the union DTO and auto-detects the type.

Path parameters:

- request_id: UUID (required)

Headers:

- SWIYU-API-Version: optional string; "2" for OID4VP 1.0/DCQL. Default is 2.

Request body: `VerificationPresentationUnionDto` as form fields (exact names, no JsonProperty remapping for form mode):

- vp_token: JSON object mapping credential query IDs array of JWT presentations

- error: `VerificationClientErrorDto` (for rejection flows)
- error_description: string (rejection reason)
- response: string (JWE for encrypted DCQL; must be the only field present among submission fields)

7.4.3.1. Example Request with dcql

```
vp_token={
    "VerifiableCredentialId": [
     "eyJ2ZXIiOiIxLjAiLCJ0eX..."
    ]
}
```

7.4.3.2. Example payload encrypted with DCQL (Only available on SWIYU-API-Version: "2")

```
response=eyJlcGsiOnsia3R5IjoiRUMiL...
```

Responses:

- **200 OK**: No body. Indicates a correct presentation by the wallet.
  This means all requested fields were provided including - if needed - a valid holder binding with correct nonce.
  A presentation can still fail due to either credential status being incorrect or no trust in the issuer can be established.
  A business verifier can also choose to fail the process later on due to values presented not matching the required business rules. For example: Age was not over 18 for a age verification.
- **400 Bad Request**: `ApiErrorDto` for invalid/incomplete submissions.
  This means the vp token could not be parsed, not all requested fields or wrong fields were provided or the holder binding was incorrect.
  This error should not occur for correct implemented good faith wallets.

External docs:

- OpenID4VP response parameters (ID2 draft): https://openid.net/specs/openid-4-verifiable-presentations-1_0-ID2.html#section-6.1

7.5. Error Handling

Common error payload: `ApiErrorDto`

- error: string (high-level code; may be null)
- error_description: string (summary; may be null)
- detail: string (`errorDetails`)

Submission errors in V1 are encoded via `VerificationErrorResponseCode`:

- `AUTHORIZATION_REQUEST_MISSING_ERROR_PARAM` for incomplete/invalid parameter combinations.
- Other codes may be used by deeper layers for credential/format issues. (See enum for full list.)

HTTP statuses:

- 200: Success for GET and POST (processing completed)
- 400: Bad Request for invalid submissions in POST
- 404: Request Object not found in GET

7.6. Field & Enum Reference

7.6.1. VPApiVersion

Value	Meaning
"2"	V1 – OID4VP 1.0 with DCQL

7.6.2. VerificationPresentationUnionDto (form fields)

- vp_token: object (V1/DCQL)
- error: VerificationClientErrorDto (rejection)
- error_description: string (rejection)
- response: string (JWE; encrypted DCQL)

7.6.3. RequestObjectDto (JSON or JWT content)

- client_id, client_id_scheme, response_type, response_mode, response_uri, nonce, version, state
- dcql_query (DCQL)
- client_metadata (inline; optional)

7.6.4. OpenidClientMetadataDto

- client_id (required)
- vp_formats (deprecated)
- vp_formats_supported (preferred)
- jwks
- encrypted_response_enc_values_supported
- additionalProperties via JsonAnySetter/Getter

7.7. Examples

7.7.1. Fetch Client Metadata

```
curl -X GET https://verifier.example.com/oid4vp/api/openid-client-metadata.json
```

7.7.2. Retrieve Request Object (JSON or JWT)

```
curl -X GET https://verifier.example.com/oid4vp/api/request-object/2f7b5c89-8c29-4f31-9d9d-7c6c0a3e6b21
```

7.7.3. Submit Rejection

```
curl -X POST \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  https://verifier.example.com/oid4vp/api/request-object/2f7b5c89-8c29-4f31-9d9d-7c6c0a3e6b21/response-data \
  -d 'error=client_rejected&error_description=The%20owner%20has%20declined%20the%20verification%20request.'
```

7.7.4. Submit V1/DCQL Presentation

```
curl -X POST \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -H 'SWIYU-API-Version: 2' \
  --data-urlencode 'vp_token={"identity_credential":["eyJhbGci..."]}' \
  https://verifier.example.com/oid4vp/api/request-object/2f7b5c89-8c29-4f31-9d9d-7c6c0a3e6b21/response-data
```

7.7.5. Submit Encrypted DCQL (JWE) (SWIYU-API-Version: 2)

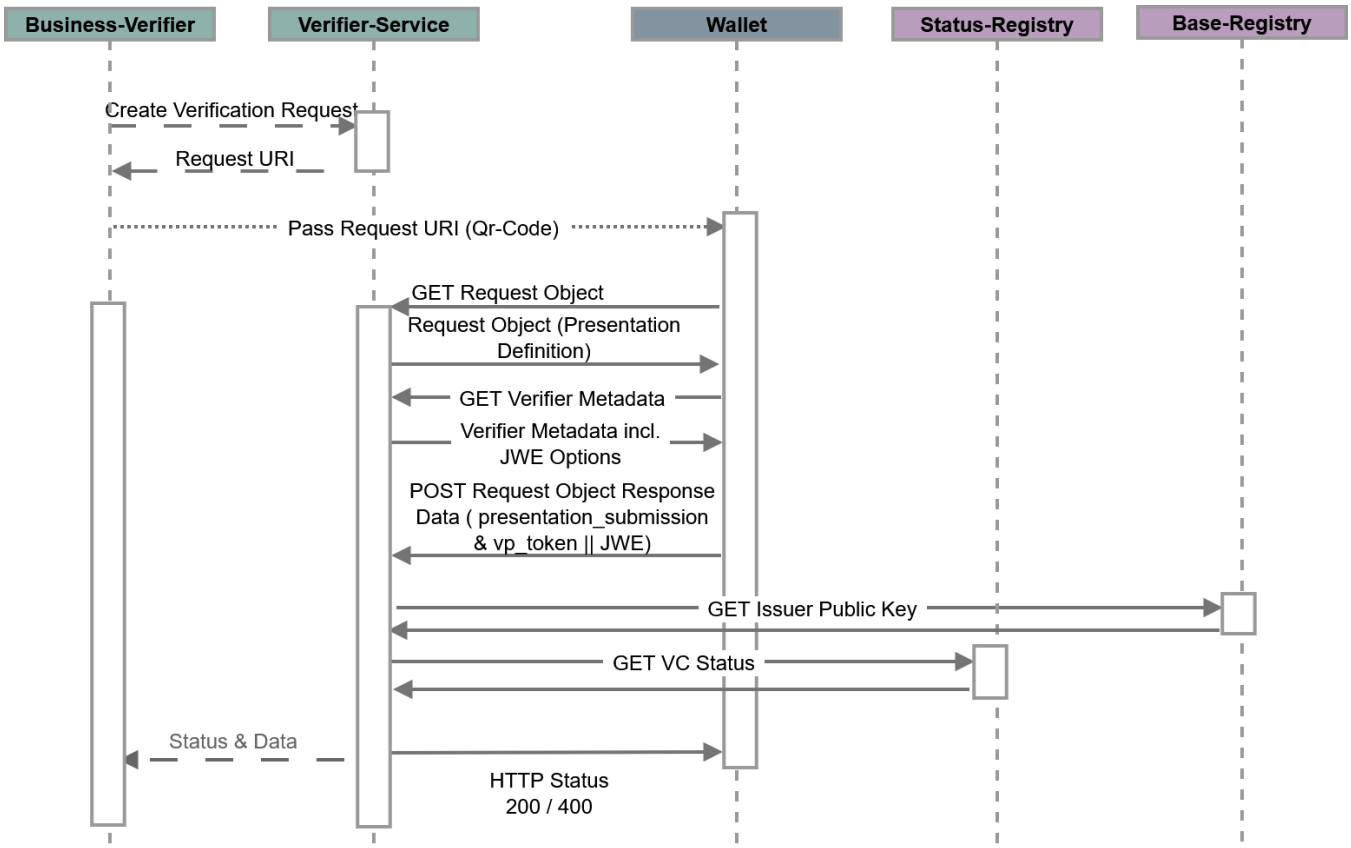
```
curl -X POST \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -H 'SWIYU-API-Version: 2' \
  -d 'response=eyJhbGciOiJSU0ExXzUiLCJlbmMiOiJBMjU2R0NNIiwidHlwIjoiSldFIn0...' \
  https://verifier.example.com/oid4vp/api/request-object/2f7b5c89-8c29-4f31-9d9d-7c6c0a3e6b21/response-data
```

8. Runntime - Verification Process - Complete

Verification Flow (cross-device flow) - Overview

The **cross-device flow** is a way for a **Verifier** (the party asking for information) to get Verifiable Credentials (VCs) from an **End-User's Wallet** when the Verifier and the Wallet are on **different devices**.

1. The Verifier asks for Credentials (using a QR code):
    - The Business-Verifier starts the process by creating an Authorization Request.
    - To make this request easy to use across different devices, the Business-Verifier usually displays it as a QR code.
    - This QR code often contains a Request URI, which is a link that points to the full request details (Deeplink). This keeps the QR code small and manageable.
2. The User's Wallet Scans and Fetches:
    - The End-User, on a separate device (like a smartphone), uses their Wallet application to scan the QR code.
    - After scanning, the Wallet sends an HTTPS GET request to the Request URI it found in the QR code. This lets the Wallet retrieve the complete Authorization Request details.
3. The Wallet Understands and Asks for User Permission:
    - The retrieved request contains a Presentation Definition. This definition clearly tells the Wallet what type of credentials the Verifier needs, in what format, and even specific pieces of information (claims) from those credentials (this is called "selective disclosure").
    - The Wallet then figures out which of the End-User's stored VCs match these requirements.
    - The Wallet also authenticates the End-User and asks for their permission (consent) to share the requested credentials. This gives the End-User control over their data.
4. The Wallet Sends the Credentials to the Verifier:
    - Once the End-User gives consent, the Wallet creates one or more Verifiable Presentations (VPs) from the selected VCs.
    - These VPs are put into something called a VP Token.
    - The Wallet then sends an Authorization Response directly to the Verifier using an HTTPS POST request. This is done using a specific Response Mode called `direct_post`.



8.1. Verification Process - Detailed

This UML sequence diagram describes the interaction flow for a **verification process** in detail

8.1.1. 1. Verification Request Creation

- The **Business Verifier** initiates the process by sending a request to the **Verifier Service** to create a verification request.
- The **Verifier Service** stores this new request in the **Verifier DB**.
- After storing the request, the Verifier Service responds back to the Business Verifier with a **Verification URI**.

8.1.2. 2. Forwarding to the Holder

- The **Business Verifier** forwards the received **Verification URI** to the **Holder** (e.g., a person or entity holding credentials).

8.1.3. 3. Holder Retrieves the Request

- The **Holder** uses the Verification URI to retrieve the request object from the **Verifier Service**.
- The Verifier Service fetches the corresponding verification request from the **Verifier DB**.

8.1.4. 4. Optional Signing

- If signing is requested:
    - **If an HSM (Hardware Security Module)** is available, the Verifier Service sends the response to the HSM for signing.
    - **If a signing key is available internally**, the Verifier Service signs the response itself.
- The signed request object is then returned to the Holder.

8.1.5. 5. Holder Sends Verifiable Presentation

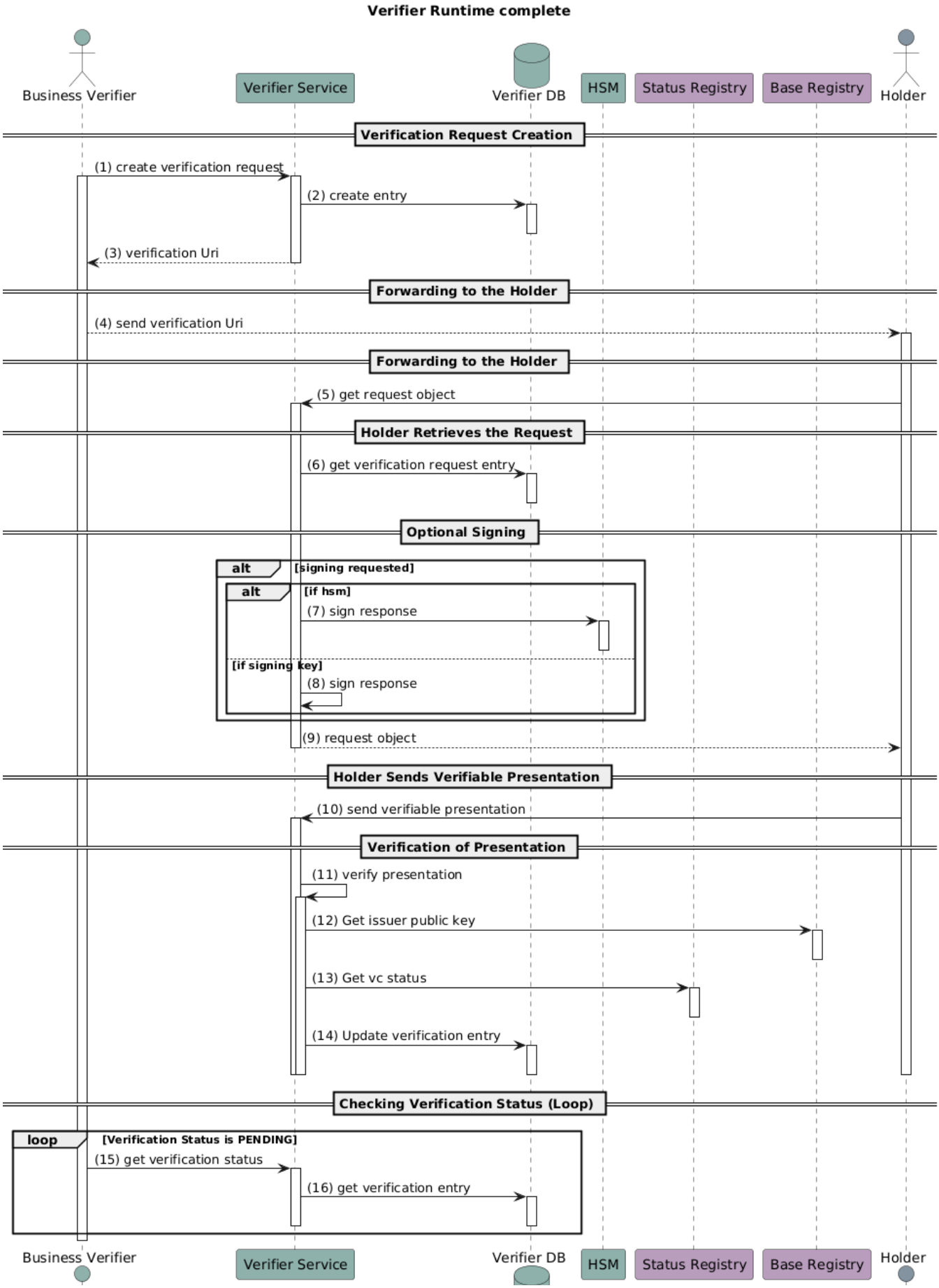
- The **Holder** sends their **Verifiable Presentation** (VP) to the Verifier Service.

8.1.6. 6. Verification of Presentation

- The **Verifier Service**:
    - Internally verifies the presentation.
    - Requests the issuer's public key from the **Base Registry** to validate the credential.
    - Checks the credential status with the **Status Registry**.
    - Updates the verification result in the **Verifier DB**.

8.1.7. 7. Checking Verification Status (Loop)

- While the verification status remains **PENDING**:
    - The **Business Verifier** periodically polls the **Verifier Service** to get the current status.
    - The Verifier Service reads the status from the **Verifier DB** and responds accordingly.

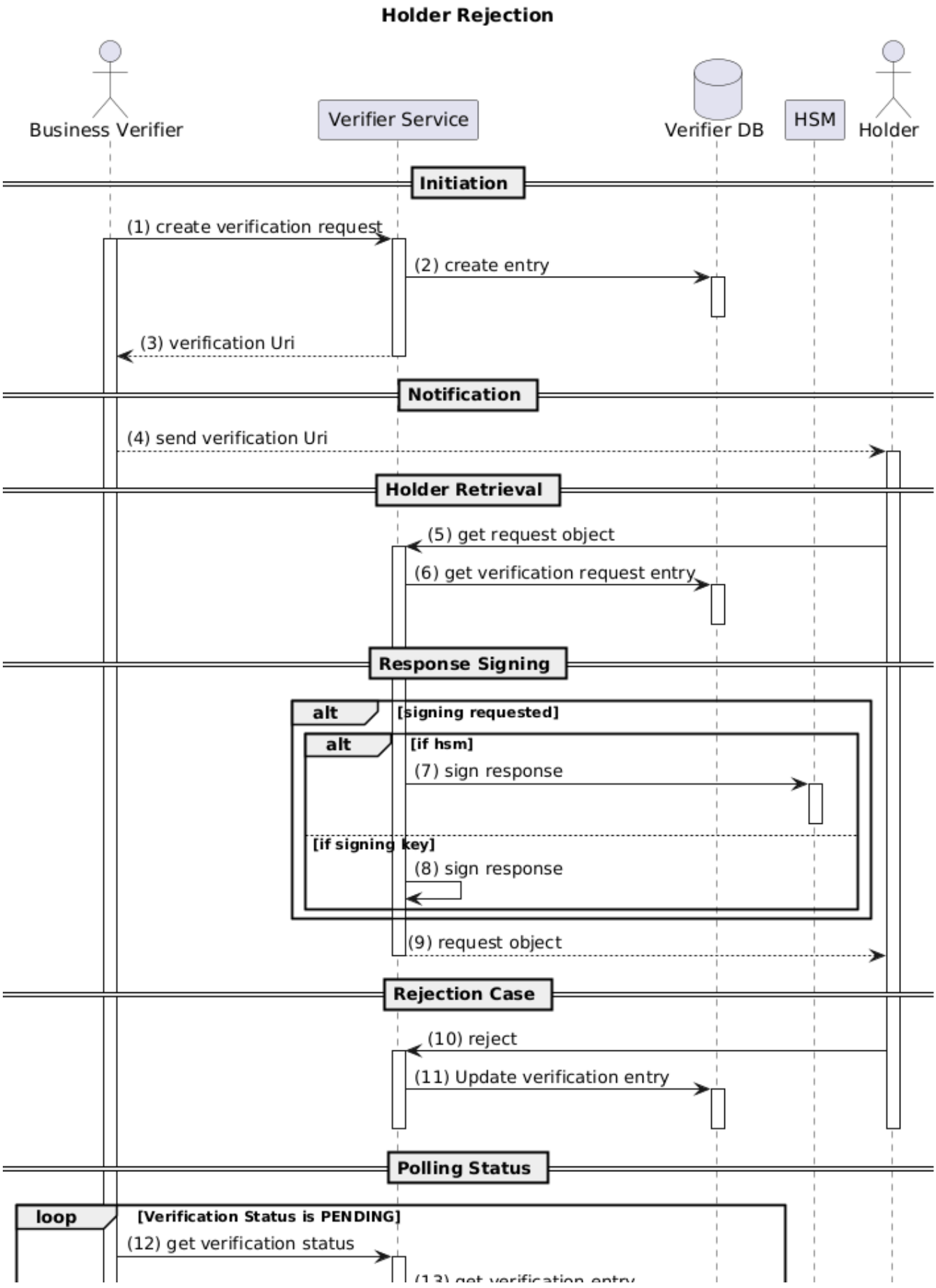

Verifier Runtime complete
Business Verifier
Verifier Service
Verifier DB
HSM
Status Registry
Base Registry
Holder
Verification Request Creation
(1) create verification request
(2) create entry
(3) verification Uri
Forwarding to the Holder
(4) send verification Uri
Forwarding to the Holder
(5) get request object
Holder Retrieves the Request
(6) get verification request entry
Optional Signing
alt
[signing requested]
alt
[if hsm]
(7) sign response
[if signing key]
(8) sign response
(9) request object
Holder Sends Verifiable Presentation
(10) send verifiable presentation
Verification of Presentation
(11) verify presentation
(12) Get issuer public key
(13) Get vc status
(14) Update verification entry
Checking Verification Status (Loop)
loop
[Verification Status is PENDING]
(15) get verification status
(16) get verification entry
Business Verifier
Verifier Service
Verifier DB
HSM
Status Registry
Base Registry
Holder

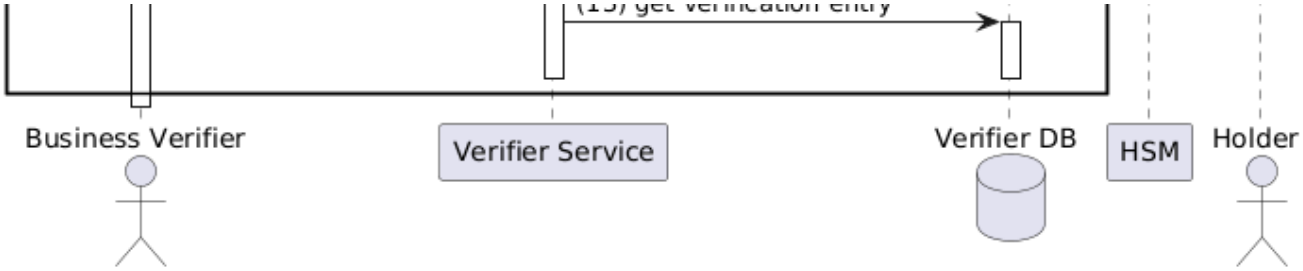
9. Runntime - Verification Process - Holder Rejection

Rejection occurs when the end-user, who controls the wallet, refuses to share their information. This is a core principle of user-centricity in OpenID for Verifiable Credentials (OpenID4VC).

- The end-user has full control over the disclosure of their information and must explicitly consent to the presentation of credentials.
- The wallet is required to obtain the end-user's consent before presenting any requested credentials.

1. **Initiation:**
    - The **Business Verifier** sends a request to the **Verifier Service** to create a verification request.
    - The **Verifier Service** stores the verification details in the **Verifier DB**.
    - It then responds to the **Business Verifier** with a **verification URI**.
2. **Notification:**
    - The **Business Verifier** forwards this verification URI to the **Holder**.
3. **Holder Retrieval:**
    - The **Holder** uses the URI to request the **verification object** from the **Verifier Service**.
    - The **Verifier Service** retrieves the corresponding entry from the **Verifier DB**.
4. **Response Signing:**
    - If signing is requested:
        - If using a **Hardware Security Module (HSM)**, the **Verifier Service** sends the response to the **HSM** for signing.
        - Otherwise, the **Verifier Service** signs it using an internal signing key.
    - The signed **verification object** is then returned to the **Holder**.
5. **Rejection Case:**
    - The **Holder** may choose to **reject** the verification.
    - The **Verifier Service** updates the corresponding entry in the **Verifier DB** to reflect the rejection.
6. **Polling Status:**
    - While the verification status is still **PENDING**, the **Business Verifier** can repeatedly query the **Verifier Service** to check the status.
    - Each time, the service retrieves the verification entry from the **DB** and responds accordingly.


Holder Rejection
Business Verifier
Verifier Service
Verifier DB
HSM
Holder
Initiation
(1) create verification request
(2) create entry
(3) verification Uri
Notification
(4) send verification Uri
Holder Retrieval
(5) get request object
(6) get verification request entry
Response Signing
alt
[signing requested]
alt
[if hsm]
(7) sign response
[if signing key]
(8) sign response
(9) request object
Rejection Case
(10) reject
(11) Update verification entry
Polling Status
loop
[Verification Status is PENDING]
(12) get verification status



10. Runntime - Trust Infrastructure Interactions

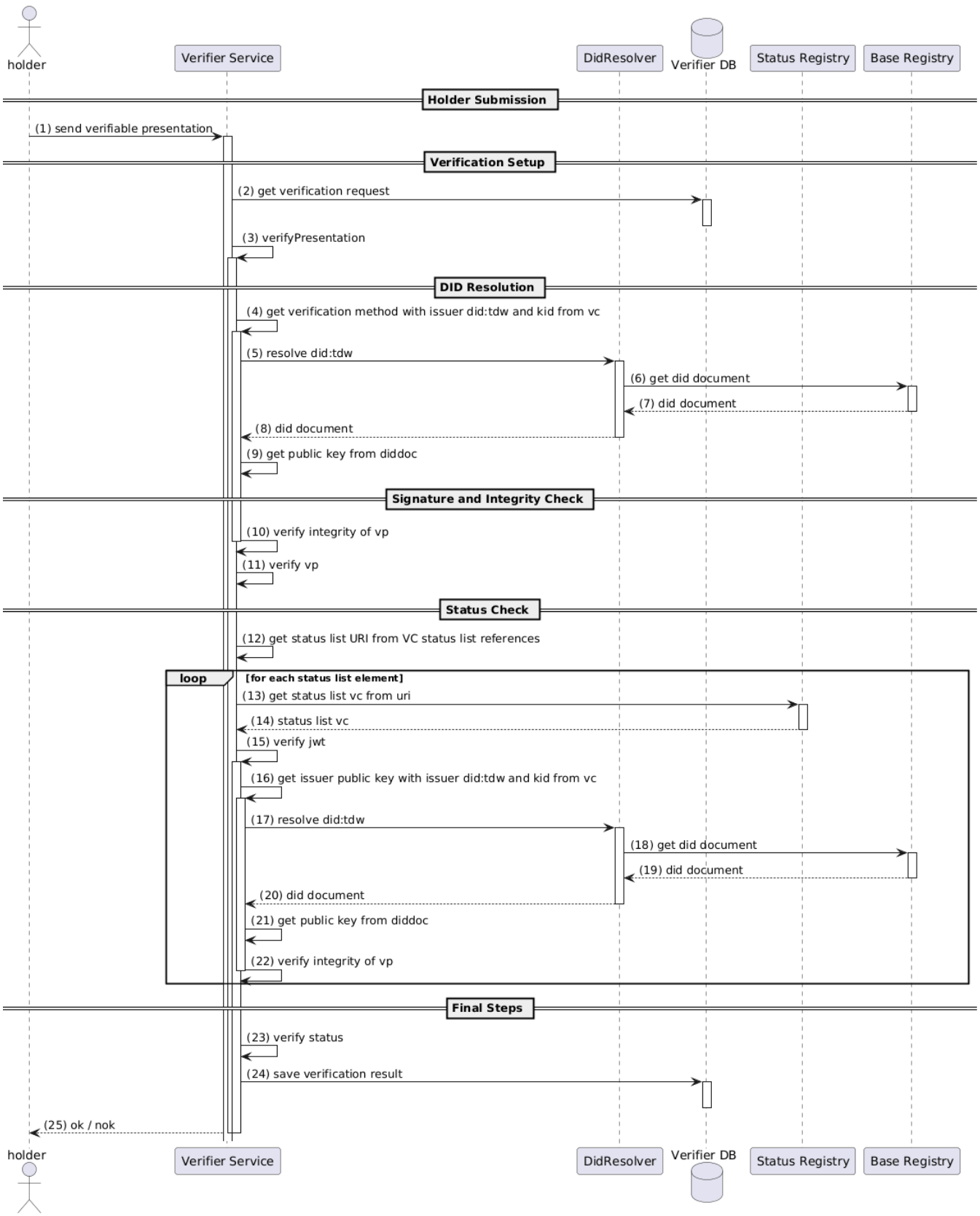
Constraint

This diagram does not contain all the interactions done in the verification process. It is limited to show the interactions between the verifier components and the trust infrastructure.

A **Status Registry** (or status register) is a service or tool that provides information about whether a **DID** (Decentralized Identifier) or a **Verifiable Credential (VC)** is currently valid.
A **Base Registry** (or basic register) is a core service or tool that provides **verifiable information** about entities, such as **issuers** or **DID controllers**.
A **Trust Registry** is an additional service to establish trustworthiness of actors. These are further described in 6.2.7 Validate Issuer via Trust Registry (TP2 Verifier-View), 6.2.8 Self-Service Flow for Verifiers to Register vqPS and 6.2.9. Sidechannel TS in the Generic Verifier (TP2 - idTS & pvaTS)

This diagram shows the **process of verifying a Verifiable Presentation (VP)** submitted by a **holder** to a **Verifier Service** (based on OpenID for Verifiable Presentations, or OID4VP). Here's a summary of the main steps:

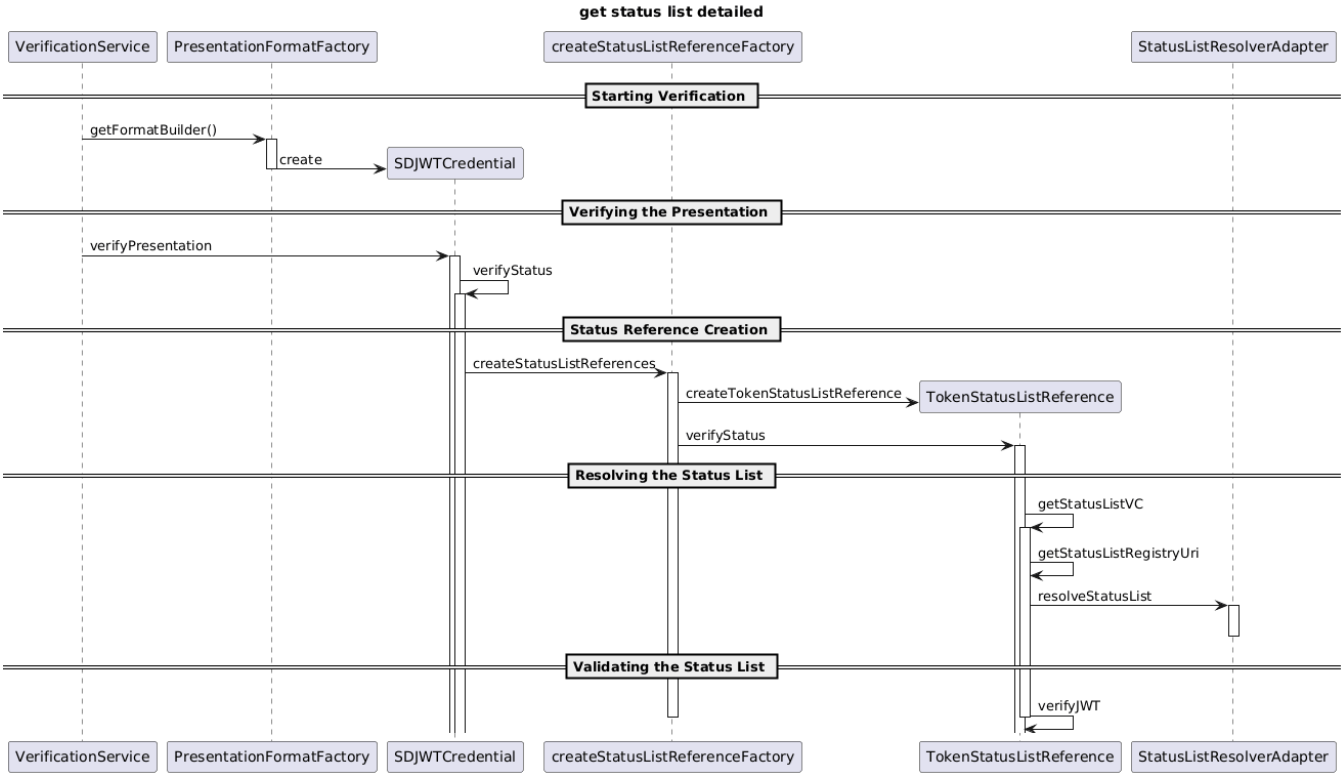
1. **Holder Submission**:
    - The **holder** sends a **Verifiable Presentation (VP)** to the **Verifier Service**.
2. **Verification Setup**:
    - The verifier looks up the related **verification request** in its **database**.
    - It starts verifying the **presentation** by checking its structure and signatures.
3. **DID Resolution**:
    - The verifier needs to verify the **issuer's identity**, so it:
        - Resolves the **DID** of the issuer via a **DID Resolver**.
        - The resolver queries the **Base Registry** to retrieve the **DID Document**, which contains the issuer's public keys.
4. **Signature and Integrity Check**:
    - The verifier uses the public key from the DID Document to verify the **VP's signature** and **integrity**.
5. **Status Check**:
    - The verifier checks the **status** of the credentials included in the VP:
        - It retrieves the **Status List VC** from the **Status Registry** using a URI provided in the credential.
        - Verifies the **JWT** and the **status** (e.g., whether the credential is revoked).
        - Performs a similar DID resolution and signature check for the Status List VC.
6. **Final Steps**:
    - The verifier determines if the presentation is valid or not.
    - It saves the **verification result** in the database.
    - It responds to the **holder** with **OK** (valid) or **NOK** (not valid).


Trust Infrastructure Interactions
holder
Verifier Service
DidResolver
Verifier DB
Status Registry
Base Registry
Holder Submission
(1) send verifiable presentation
Verification Setup
(2) get verification request
(3) verifyPresentation
DID Resolution
(4) get verification method with issuer did:tdw and kid from vc
(5) resolve did:tdw
(6) get did document
(7) did document
(8) did document
(9) get public key from diddoc
Signature and Integrity Check
(10) verify integrity of vp
(11) verify vp
Status Check
(12) get status list URI from VC status list references
loop
[for each status list element]
(13) get status list vc from uri
(14) status list vc
(15) verify jwt
(16) get issuer public key with issuer did:tdw and kid from vc
(17) resolve did:tdw
(18) get did document
(19) did document
(20) did document
(21) get public key from diddoc
(22) verify integrity of vp
Final Steps
(23) verify status
(24) save verification result
(25) ok / nok
holder
Verifier Service
DidResolver
Verifier DB
Status Registry
Base Registry

11. Runntime - Get Status list - Detailed

This diagram shows the **internal verification process** used by a **Verification Service** to check the validity and status of a **Verifiable Presentation**, specifically using the **SD-JWT (Selective Disclosure JWT) credential format**. Here's a simple explanation of what happens:

1. **Starting Verification**:
    - The **VerificationService** asks the **PresentationFormatFactory** for a builder that can handle the required credential format (in this case, SD-JWT).
    - The factory creates an **SDJWTCredential** object, which is responsible for handling the SD-JWT presentation format.
2. **Verifying the Presentation**:
    - The **VerificationService** tells the **SDJWTCredential** to verify the presentation.
    - As part of that process, it also checks the **status** of the credentials (e.g., to see if they are revoked).
3. **Status Reference Creation**:
    - A **StatusListReferenceFactory** is used to create a **TokenStatusListReference** object, which will manage the status list for the credentials.
4. **Resolving the Status List**:
    - The **TokenStatusListReference** determines the URI of the relevant **Status List VC**.
    - It uses a **StatusListResolverAdapter** to fetch the **Status List** from the appropriate registry or source.
5. **Validating the Status List**:
    - Once retrieved, the **Status List VC** is checked:
        - The JWT is verified to ensure integrity and authenticity.
        - The **status** (e.g., revoked or valid) of the credential is confirmed.



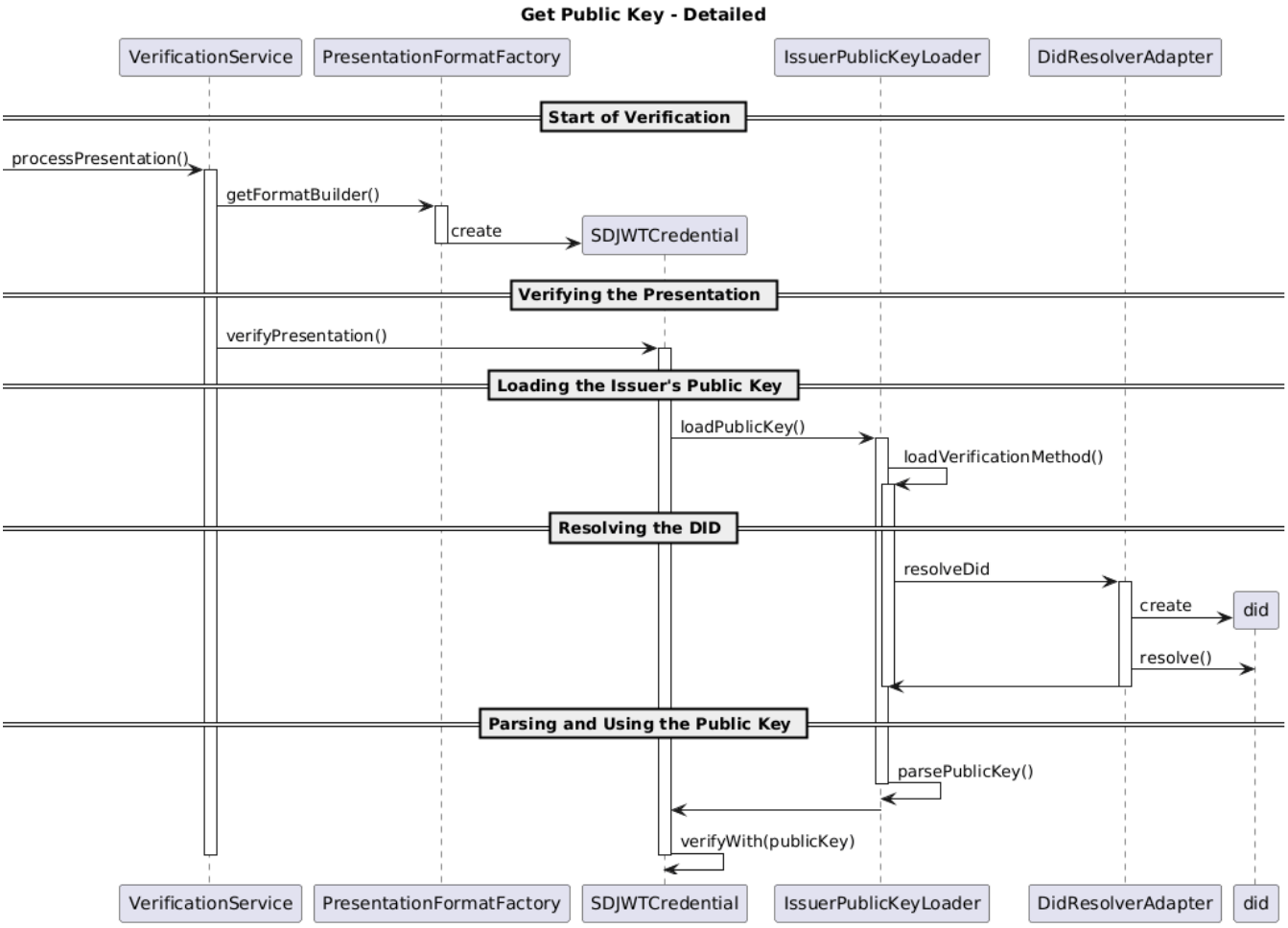
12. Runntime - Get Public Key - Detailed

This diagram shows the **process of verifying a Verifiable Presentation** using an **SD-JWT credential**, focusing on how the **Verification Service** retrieves the issuer's public key and verifies the signature. Here's a simplified explanation:

1. **Start of Verification**:
    - A presentation is received and the **VerificationService** begins processing it (`processPresentation()`).
    - The service asks the **PresentationFormatFactory** for a format handler.
    - The factory creates an **SDJWTCredential** object to manage the SD-JWT format.
2. **Verifying the Presentation**:

- The **VerificationService** calls the `verifyPresentation()` method on the **SDJWTCredential** object.

3. **Loading the Issuer's Public Key**:
    - To verify the signature, the **SDJWTCredential** needs the issuer's public key.
    - It delegates this task to an **IssuerPublicKeyLoader**.
    - The loader retrieves the **verification method** (typically from a DID document).
4. **Resolving the DID**:
    - The **IssuerPublicKeyLoader** uses a **DidResolverAdapter** to resolve the issuer's DID.
    - The adapter creates a DID object and resolves it (fetches the DID Document).
5. **Parsing and Using the Public Key**:
    - Once the DID Document is received, the **IssuerPublicKeyLoader** parses the public key.
    - The **SDJWTCredential** uses this key to verify the **signature** of the presentation.



13. Runntime - Implementing DCQL in Generic Verifier Service

Previously, OpenID for Verifiable Presentations (OIDC4VP) integrated the **DIF Presentation Exchange (PE) specification** into the "claims" request parameter as the query language for Verifier requirements for Credentials.

However, the specification has significantly evolved. A **newer draft (Draft 29) of OpenID for Verifiable Presentations** introduces a new, more explicit, and flexible query language: the **Digital Credentials Query Language (DCQL)**.

The main reasons for this switch to DCQL are:

- **Be compliance with the new standard 1.0**
- **Increased Flexibility and Ease of Use**: DCQL is defined as a **JSON-encoded query language**, enabling presentations to be **requested in a simpler and more flexible manner**.
- **Improved Granularity and Data Privacy**: DCQL allows the Verifier to **precisely specify the requirements for Credentials and individual claims**. This is crucial for **selective disclosure**, where only the strictly necessary information is shared, significantly **enhancing user privacy**.

Verifiers are explicitly encouraged to use DCQL queries that request only the **minimal amount of claims and Credentials** required to fulfill the specified purposes.

- **Support for Complex Scenarios**: DCQL can encode **constraints for combinations of Credentials and claims** or express various alternatives for how a specific Credential can satisfy a request.

Example 1: "PID" Credential Alternatives A Verifier might request a "pid" credential, but allow for alternatives if that specific one isn't available. The query could specify:

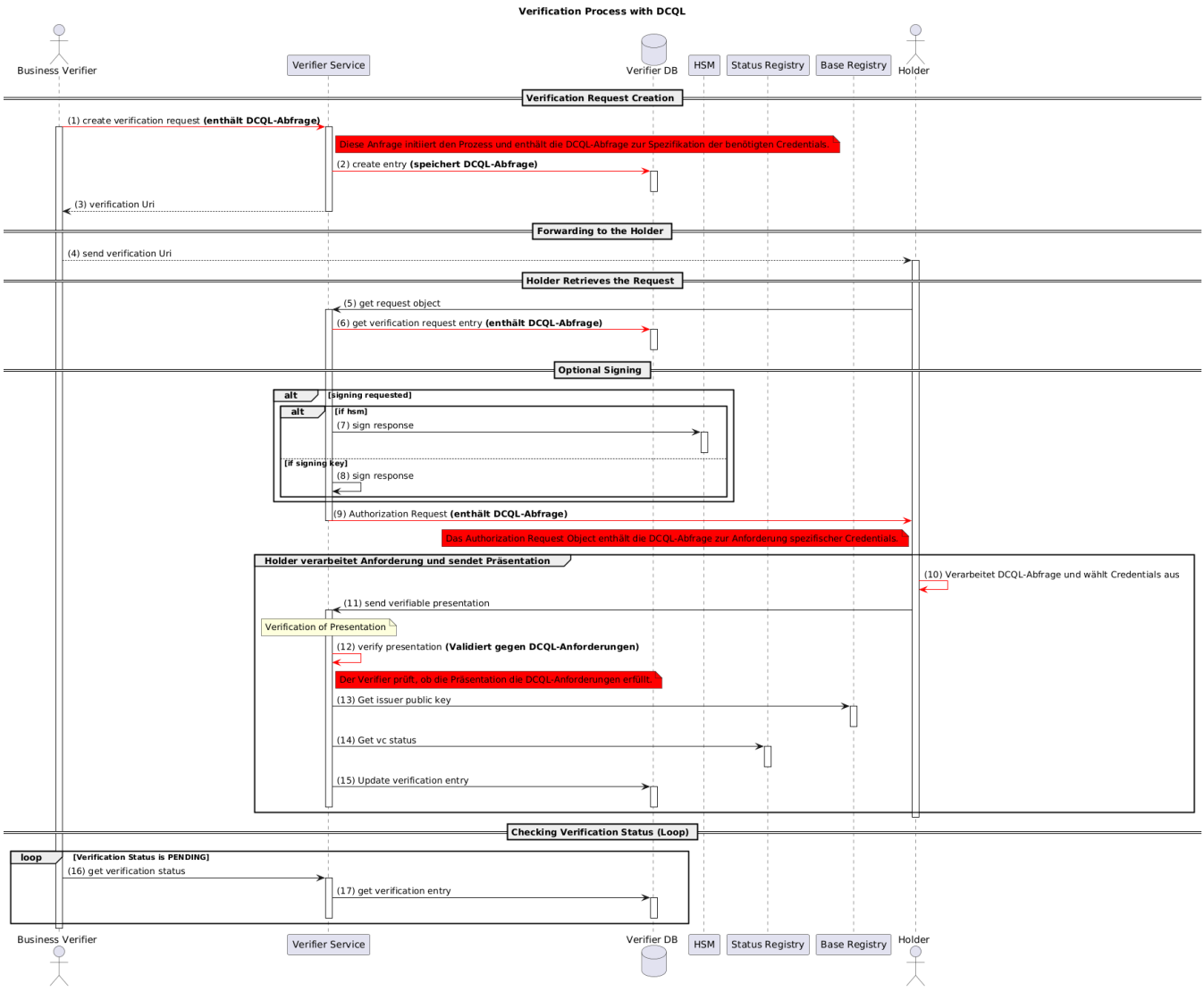
The `pid` Credential, OR

The `other_pid` Credential, OR

A combination of both `pid_reduced_cred_1` AND `pid_reduced_cred_2` This shows how DCQL can handle disjunctive (`OR`) and conjunctive (`AND`) logic across multiple credentials.

Example 2: Identity and Address from different sources A query might ask for an ID and an address, where *either* can come from an mDL (mobile Driving License) *or* a photo ID card. This demonstrates flexibility in sourcing the required information from different types of credentials.

This UML sequence diagram describes the interaction flow for a verification process in detail:



13.1.1. 1. Verification Request Creation

- The **Business Verifier** initiates the process by sending a request to the **Verifier Service** to create a verification request. (This request **now contains a DCQL query**)
- The **Verifier Service** stores this new request, including the DCQL query, in the **Verifier DB**.

- After storing the request, the Verifier Service responds back to the Business Verifier with a **Verification URI**.

13.1.2. 2. Forwarding to the Holder

- The **Business Verifier** forwards the received **Verification URI** to the **Holder** (e.g., a person or entity holding credentials).

13.1.3. 3. Holder Retrieves the Request

- The **Holder** uses the Verification URI to retrieve the request object from the **Verifier Service**. This request object **contains the DCQL query**, which describes the Verifier's requirements for the Credentials to be presented, including the type, format, and individual claims (selective disclosure). The Holder's Wallet processes this DCQL query to identify suitable Credentials and obtain the End-User's consent for the presentation.
- The Verifier Service fetches the corresponding verification request from the **Verifier DB**.

13.1.4. 4. Optional Signing

- If signing is requested:
    - **If an HSM (Hardware Security Module)** is available, the Verifier Service is using a key on HSM for signing.
    - **If a signing key is available internally**, the Verifier Service signs the response itself.
- The signed request object is then returned to the Holder.

13.1.5. 5. Holder Sends Verifiable Presentation

- The **Holder** sends their **Verifiable Presentation** (VP) to the Verifier Service.

13.1.6. 6. Verification of Presentation

- The **Verifier Service**:
    - Internally verifies the presentation.
    - The **Verifier must ensure that the returned Credentials meet all criteria defined in the DCQL query of the authorization request**
    - Requests the issuer's public key from the **Base Registry** to validate the credential if not already done
    - Checks the credential status with the **Status Registry**.
    - Updates the verification result in the **Verifier DB**.

13.1.7. 7. Checking Verification Status (Loop)

- While the verification status remains **PENDING**:
    - The **Business Verifier** periodically polls the **Verifier Service** to get the current status or will be informed with Webhooks.
    - The Verifier Service reads the status from the **Verifier DB** and responds accordingly.

13.2. Changes in Interface for switching from DIF Presentation Exchange (DIF PE) to **Digital Credentials Query Language (DCQL)**

The **Digital Credentials Query Language (DCQL)** replaces the earlier **DIF Presentation Exchange (PE)** specification as the query language for verifier requirements. This change is made to ensuring compliance with the **OpenID for Verifiable Presentations (OID4VP) Standard 1.0**.
The main adjustments to the verifier service interfaces include the parallel introduction of the `dcql_query` parameter as well as support for versioning headers and new structures for Verifiable Presentations.

13.2.1. 1. `/management/api/verifications` (POST) – Create a Verification Request

- **Purpose:** Creates a new verification process with the specified attributes.
- **Request Schema (CreateVerificationManagement):**
    - `presentation_definition`: References `#/components/schemas/PresentationDefinition`.
      Description: *"Presentation definition according to https://identity.foundation/presentation-exchange/#presentation-definition".*
      This parameter is optional.
    - `dcql_query`: References `#/components/schemas/DcqlQueryDto`.
      This parameter is optional.
        - **Validation Note (from concept):** API validation must ensure that **either** `presentation_definition` **OR** `dcql_query` is provided in a request. It is also allowed to send both.
- **Response Schema (ManagementResponse):**
    - `presentation_definition`: References `#/components/schemas/PresentationDefinition`. Optional.
    - `dcql_query`: References `#/components/schemas/DcqlQueryDto`. Optional.

13.2.2. 2. `/management/api/verifications/{verificationId}` (GET) – Retrieve Verification Status

- **Purpose:** Retrieves a verification process by its ID.
- **Response Schema (ManagementResponse):**
    - `presentation_definition`: References `#/components/schemas/PresentationDefinition`. Optional.
    - `dcql_query`: References `#/components/schemas/DcqlQueryDto`. Optional.

13.2.3. 3. `/oid4vp/api/request-object/{request_id}` (GET) – Retrieve Request Object by Holder

- **Purpose:** Returns a `RequestObjectDto` either as a JSON object or as a signed JWT string, depending on the JAR (JWT secured authorization request) flag in the verifier management.
- **Response Schema (RequestObject):**
    - `presentation_definition`: References `#/components/schemas/PresentationDefinition`.
      Description: *"Presentation definition according to https://identity.foundation/presentation-exchange/#presentation-definition.*
      This field is only used for requests initiated with the older Presentation Exchange (PE) format."
      Optional.
    - `dcql_query`: References `#/components/schemas/DcqlQueryDto`.
      Description: *"DCQL query object as an Authorization Request parameter."*
      Optional.

13.2.4. 4. `/oid4vp/api/request-object/{request_id}/response-data` (POST) – Receive Verifiable Presentation

- **Purpose:** Processes different types of verification presentations, including standard presentations, rejections, DCQL presentations, and encrypted DCQL presentations. The method automatically determines the request type based on the provided parameters.
- **Request Headers:**
    - `SWIYU-API-Version`
        - Description: Optional API version header.
        - Type: String.
        - Optional.
        - Supported values (enum):
            - `"1"`: Supports OID4VP ID2 with DIF Presentation Exchange.
            - `"2"`: Supports OID4VP 1.0.
- **Request Schema (VerificationPresentationUnion):**
    - `vp_token`:
        - Description: *"VP token that can be either a string for standard presentations or a JSON object for DCQL presentations.*
          For standard/PE presentations: JWT token string.
          For DCQL presentations: Object containing credential query results where keys are query IDs and values are arrays of presentations."
        - Example:
            - Standard/PE: `"eyJhbGci...QMA"`
            - DCQL: `{"my_credential": ["eyJhbGci...QMA", "eyJhbGci...QMA"]}`
        - Type: `oneOf` with `{"type": "string"}` and `{}` (arbitrary JSON object for DCQL presentations).
    - `presentation_submission`:
        - Type: String.
        - Description: *"The presentation submission as defined in DIF presentation submission (used for Standard and PE presentations)"*.
        - Example: `{"id":"a30e3b91-fb77-4d22-95fa-871689c322e2","definition_id":"32f54163-7166-48f1-93d8-ff217bdb0653"}`
    - `error`, `error_description`: String parameters used for request rejection.
    - `response`:
        - Type: String.
        - Description: *"Encrypted response JWE string (used for DCQLE)"*.
        - Example: `"eyJhbGciOiJSU0ExXzUiLCJlbmMiOiJBMjU2R0NNIiwidHlwIjoiSldFIn0..."`.
- **VP Token Response Encryption:**
  The Swiss profile mandates encryption of the VP token response after a transition period.
  The verifier uses its `client_metadata` to provide the wallet with the required information, including `jwks` (only P-256 keys supported) and `encrypted_response_enc_values_supported` (only A128GCM).
  A new P-256 key must be generated for every authorization request.

13.2.5. DCQL Structure (`DcqlQueryDto`) in components/schemas

The specification defines `DcqlQueryDto` as a JSON-encoded query object.

- **DcqlQueryDto**

- Type: Object.
- Description: *"Represents the Digital Credentials Query Language (DCQL) query. According to OpenID for Verifiable Presentations 1.0, Section 6."*
- Properties:
  - `credentials`:
    - Type: Array.
    - Required (minItems: 1).
    - Description: *"A required non-empty array of Credential Query objects. According to OID4VP 1.0, Section 6, property 'credentials'."*
    - Items: Reference `#/components/schemas/DcqlCredentialDto`.
  - `credential_sets`:
    - Type: Array.
    - Optional (minItems: 1).
    - Description: *"An optional non-empty array of Credential Set Query objects. According to OID4VP 1.0, Section 6, property 'credential_sets'."*
    - Items: Array of strings.

13.2.5.1. Nested Schema: `DcqlCredentialDto` (Credential Query)

- **DcqlCredentialDto**
  - Type: Object.
  - Description: *"Represents an individual Credential Query object within the 'credentials' array of a DCQL query. According to OID4VP 1.0, Section 6.1."*
  - Properties:
    - `id`: String, required. *"Uniquely identifies the credential in the response and for credential_sets."*
    - `format`: String, required. *"Specifies the format of the requested credential."*
    - `multiple`: Boolean, optional. Default = `false`.
    - `meta`: References `#/components/schemas/DcqlCredentialMetaDto`, required.
    - `claims`: Array, optional (minItems: 1). Items reference `#/components/schemas/DcqlClaimDto`.
    - `claim_sets`: Array, optional (minItems: 1).
    - `require_cryptographic_holder_binding`: Boolean, optional. Default = `true`.
    - `trusted_authorities`: Array, optional.
  - Required: `id`, `format`, `meta`.

13.2.5.2. Nested Schema: `DcqlCredentialMetaDto`

- **DcqlCredentialMetaDto**
  - Type: Object.
  - Description: *"Represents metadata parameters within a Credential Query according to OID4VP 1.0, Section 6.1."*
  - Properties: Defined per credential format (e.g., `vct_values` in the spec).

13.2.5.3. Nested Schema: `DcqlClaimDto` (Claims Query)

- **DcqlClaimDto**
  - Type: Object.
  - Description: *"Represents an individual claim object within the 'claims' array of a Credential Query. According to OID4VP 1.0, Section 6.3."*
  - Properties:
    - `id`: String. Required if `claim_sets` is present; otherwise optional.
    - `path`: Array of strings, required. *"Claim Path Pointer referencing the claim within the credential."*
    - `values`: Array (strings, integers, or booleans), optional. Expected values for the claim.
  - Required: `path`.

13.3. Example

SD-JWT VC Issued	Current Presentation Request (DIF format)	Future Presentation Request (DCQL format)
This represents an example of an SD-JWT Verifiable Credential that has been issued to a Holder. It includes claims and properties relevant to its structure, such as selective disclosure digests (_sd) and a confirmation method (cnf) for holder binding. Example SD-JWT VC:		This represents a Verifier's request for Credentials using the Digital Credentials Query Language (DCQL) format, the newer, more explicit, and flexible query language replacing DIF PE in later drafts of OpenID for Verifiable Presentations. This example requests two types of credentials: an identity_credential_dcql and a university_degree_dcql, specifying their formats and required claims. It also demonstrates the use of credential_sets for alternatives. Example DCQL `{`

```
{
  "_sd": [

"3oUCnaKt7wqDKuyh-
LgQozzfhgb8gO5Ni
-RCWsWW2vA",

"8z8z9X9jUtb99gj
ejCwFAGz4aqlHf-
sCqQ6eM_qmpUQ",

"Cxq4872UXXngGUL
T_kl8fdwVFkyK6AJ
fPZLy7L5_0kI",

"TGf4oLbgwd5JQaH
yKVQZU9UdGE0w5rt
DsrZzfUaomLo",

"jsu9yVulwQQlhFl
M_3JlzMaSFzglhQG
0DpfayQwLUK4",
    "sFcViHN-
JG3eTUyBmU4fkwus
y5I1SLBhe1jNvKxP
5xM",

"tiTngp9_jhC389U
P8_k67MXqoSfiHq3
iK6o9un4we_Y",
    "xsKkGJXD1-
e3I9zj0YyKNv-
lU5YqhsEAF9NhOr8
xga4"
  ],
  "iss":
"https://example
.com/issuer",
  "iat":
1683000000,
  "exp":
1883000000,
  "vct":
"https://credent
ials.example.com
/identity_creden
tial",
  "_sd_alg":
"sha-256",
  "cnf": {
    "jwk": {
      "kty":
"EC",
      "crv": "P-
256",
      "x":
"TCAER19Zvu3OHF4
j4W4vfSVoHIP1ILi
lDls7vCeGemc",
      "y":
"ZxjiWWbZMQGHVWK
VQ4hbSIirsVfuecC
E6t4jT9F2HZQ"
    }
  }
}
```


This represents a Verifier's request for Credentials using the DIF Presentation Exchange (PE) format, which was previously integrated into the OpenID for Verifiable Presentations (OIDC4VP) "claims" request parameter. This example requests an IDCardCredential using ldp_vc format.

Example DIF presentation_definition:

```
{
  "id": "vp token example",
  "input_descriptors": [
    {
      "id": "id card credential",
      "format": {
        "ldp_vc": {
          "proof_type": [

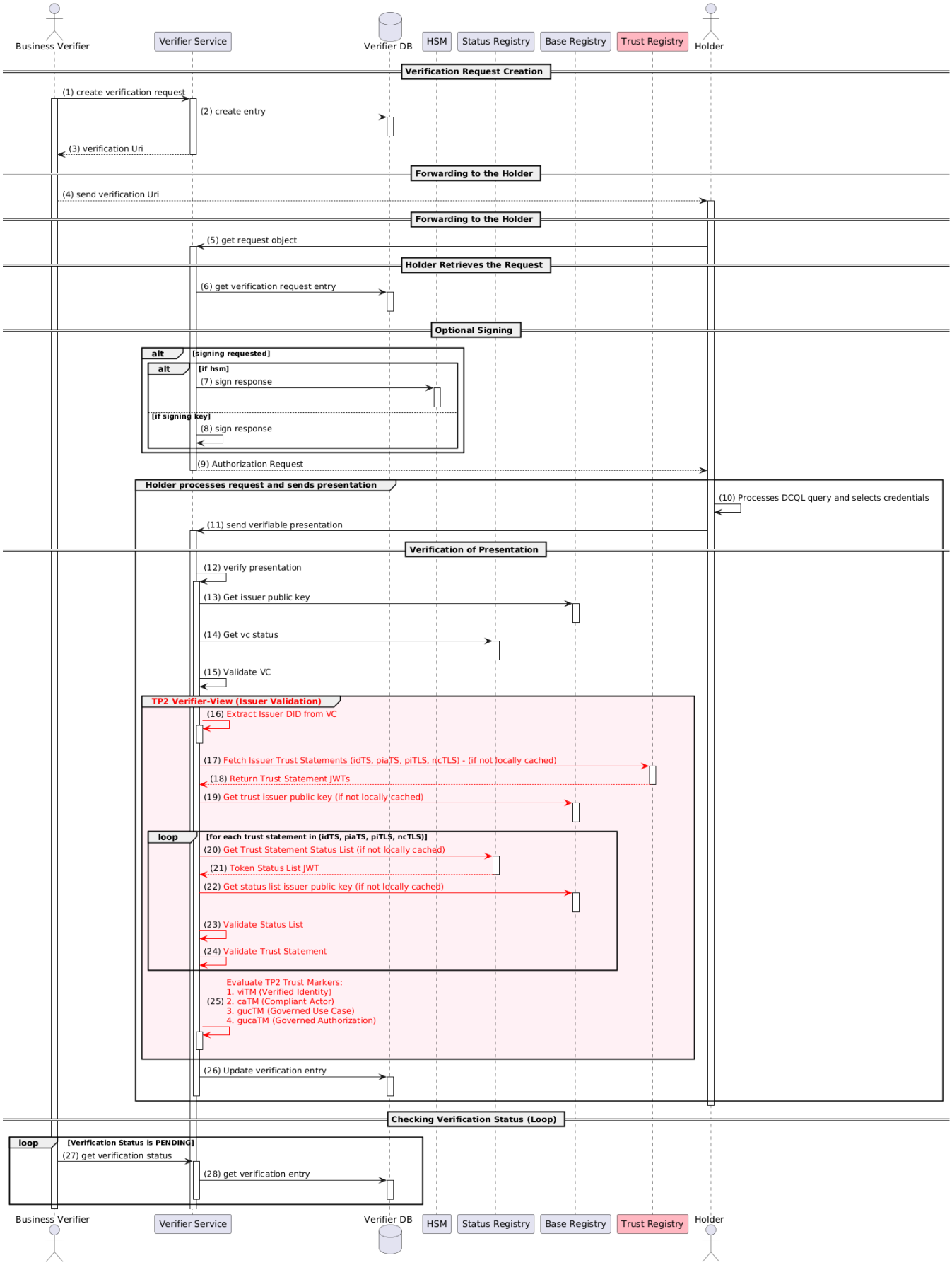
"Ed25519Signature2018"
          ]
        }
      },
      "constraints": {
        "fields": [
          {
            "path": [
              "$.type"
            ],
            "filter": {
              "type": "string",
              "pattern":
"IDCardCredential"
            }
          }
        ]
      }
    }
  ]
}
```

```
  "dcql_query": {
    "credentials": [
      {
        "id": "identity_credential_dcql",
        "format": "dc+sd-jwt",
        "meta": {
          "vct_values": [
            "https://credentials.example.com
/identity_credential"
          ]
        },
        "claims": [
          {
            "path": [
              "given_name"
            ]
          },
          {
            "path": [
              "family_name"
            ]
          },
          {
            "path": [
              "address",
              "street_address"
            ]
          }
        ],
        "require_cryptographic_holder_binding": true
      },
      {
        "id": "university_degree_dcql",
        "format": "dc+sd-jwt",
        "meta": {
          "vct_values": [
            "https://credentials.example.com
/university_degree"
          ]
        },
        "claims": [
          {
            "path": [
              "degree_name"
            ]
          },
          {
            "path": [
              "graduation_year"
            ]
          }
        ]
      }
    ],
    "credential_sets": [
      {
        "options": [
          [
            "identity_credential_dcql"
          ],
          [
            "university_degree_dcql"
          ]
        ],
        "required": true
      }
    ]
  },
  "jwt_secured_authorization_request": true
}
```

14. Runntime - Validate Issuer via Trust Registry (TP2 Verifier-View)

14.1. Sequence Diagram Verifier-View

In the following diagram the interaction using the verification of issuers via trust registry is shown. Other options of establishing trust are omitted in this view. The validation of trust is highlighted in red.


Business Verifier
Verifier Service
Verifier DB
HSM
Status Registry
Base Registry
Trust Registry
Holder
Verification Request Creation
(1) create verification request
(2) create entry
(3) verification Uri
Forwarding to the Holder
(4) send verification Uri
Forwarding to the Holder
(5) get request object
Holder Retrieves the Request
(6) get verification request entry
Optional Signing
alt
[signing requested]
alt
[if hsm]
(7) sign response
[if signing key]
(8) sign response
(9) Authorization Request
Holder processes request and sends presentation
(10) Processes DCQL query and selects credentials
(11) send verifiable presentation
Verification of Presentation
(12) verify presentation
(13) Get issuer public key
(14) Get vc status
(15) Validate VC
TP2 Verifier-View (Issuer Validation)
(16) Extract Issuer DID from VC
(17) Fetch Issuer Trust Statements (idTS, piaTS, piTLS, ncTLS) - (if not locally cached)
(18) Return Trust Statement JWTs
(19) Get trust issuer public key (if not locally cached)
loop
[for each trust statement in (idTS, piaTS, piTLS, ncTLS)]
(20) Get Trust Statement Status List (if not locally cached)
(21) Token Status List JWT
(22) Get status list issuer public key (if not locally cached)
(23) Validate Status List
(24) Validate Trust Statement
Evaluate TP2 Trust Markers:
1. viTM (Verified Identity)
(25) 2. caTM (Compliant Actor)
3. gucTM (Governed Use Case)
4. gucaTM (Governed Authorization)
(26) Update verification entry
Checking Verification Status (Loop)
loop
[Verification Status is PENDING]
(27) get verification status
(28) get verification entry

14.2. Caching for TP2 Verifier-View

In the Trust Protocol 2.0 (TP2) for the swiyu ecosystem, the **Generic Verifier** can validate the legitimacy of an issuer when processing a Verifiable Credential (VC). This process requires evaluating multiple Trust Statements. As these statements can require some bandwidth to fetch, caching is used to prevent repeatedly loading the same data.
All trust statements and their associated status lists are cached according to their expiry (`exp` claim in JWTs) or time-to-live (`ttl` claim in Token Status Lists Tokens).

14.3. Evaluate TP2 Trust Markers

Trust Markers are set according to Trust Protocol 2.0 specification. Following is an overview of what this entails.

14.3.1. Phase 1: Base Validation of Trust Statements

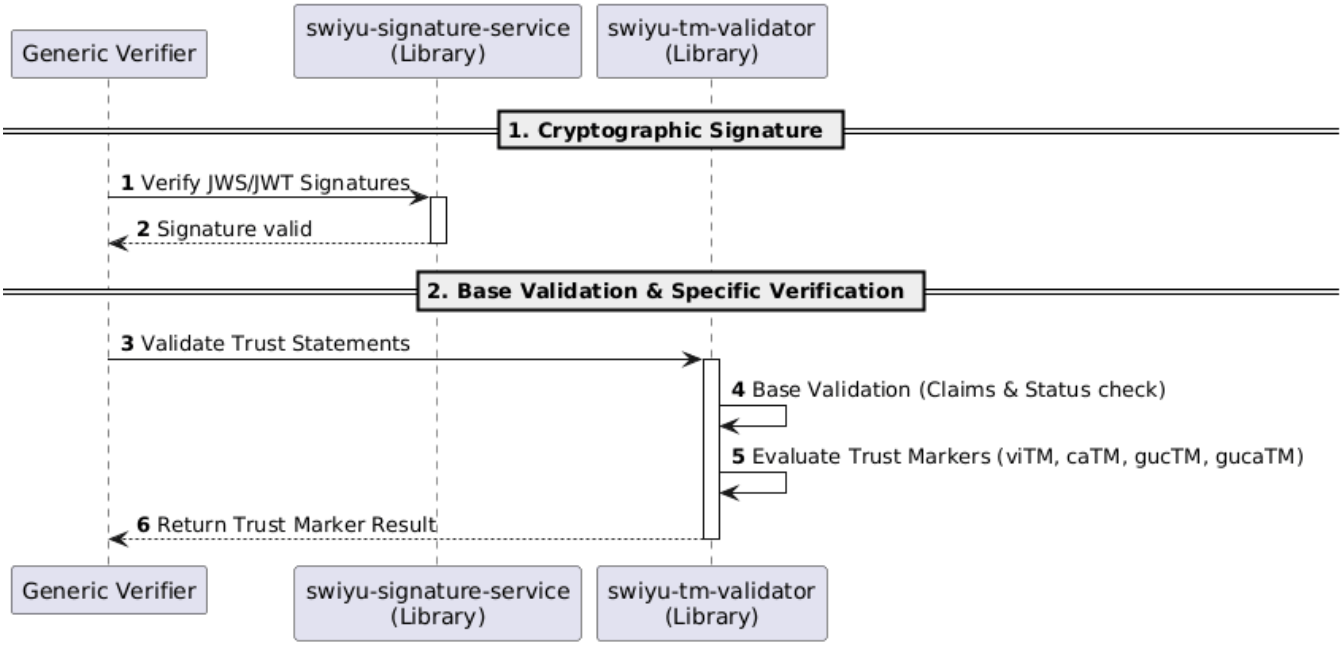
Before any Trust Marker can be evaluated, the Generic Verifier MUST ensure that the underlying Trust Statements (JWTs) retrieved from the cache or the Trust Registry are cryptographically sound and active. A statement MUST be considered **invalid** if any of the following checks fail:

- **Format & Signatures:** The JWS signature is valid, and the header claims `typ`, `alg`, and `kid` are present and correct.
- **Profile Version:** The `profile_version` claim MUST start with `swiss-profile-trust:` and follow semantic versioning.
- **Lifecycle:** The current time MUST fall within the validity period defined by `nbf` (not before) and `exp` (expiration).
- **Revocation Status:** The `status` claim MUST resolve successfully. The statement is invalid if it has been revoked or if the referenced status list was issued by a different identifier than the statement itself. (TODO: Not part of lib)

14.3.2. Phase 2: Specific Verification of Trust Markers

Once the statements are validated, the Verifier evaluates the Trust Markers for the current transaction without ever contacting the Issuer directly.

1. **viTM (Verified Identity):** Asserts that the Trust Registry provides a valid Identity Trust Statement (`idTS`) for the Issuer DID extracted from the VC.
2. **caTM (Compliant Actor):** Asserts that the Issuer is not listed as a known bad actor in the currently active Non-Compliance Trust List Statement (`ncTLS`).
3. **gucTM (Governed Use Case):** Determines if the presented VC is a protected credential by checking if its `vct` (VC Type) is listed in the active Protected Issuance Trust List Statement (`piTLS`).
4. **gucaTM (Governed Authorization):** If the VC is protected (`gucTM` == true), the Verifier MUST confirm that the Issuer has the explicit permission to issue it. This requires matching the `sub` claim of the Issuer's Protected Issuance Authorization Trust Statement (`piaTS`) with the `idTS` subject, and verifying that `piaTS.can_issue.vct` exactly matches the VC's type.



15. Runntime - Self-Service Flow for Verifiers to Register vqPS

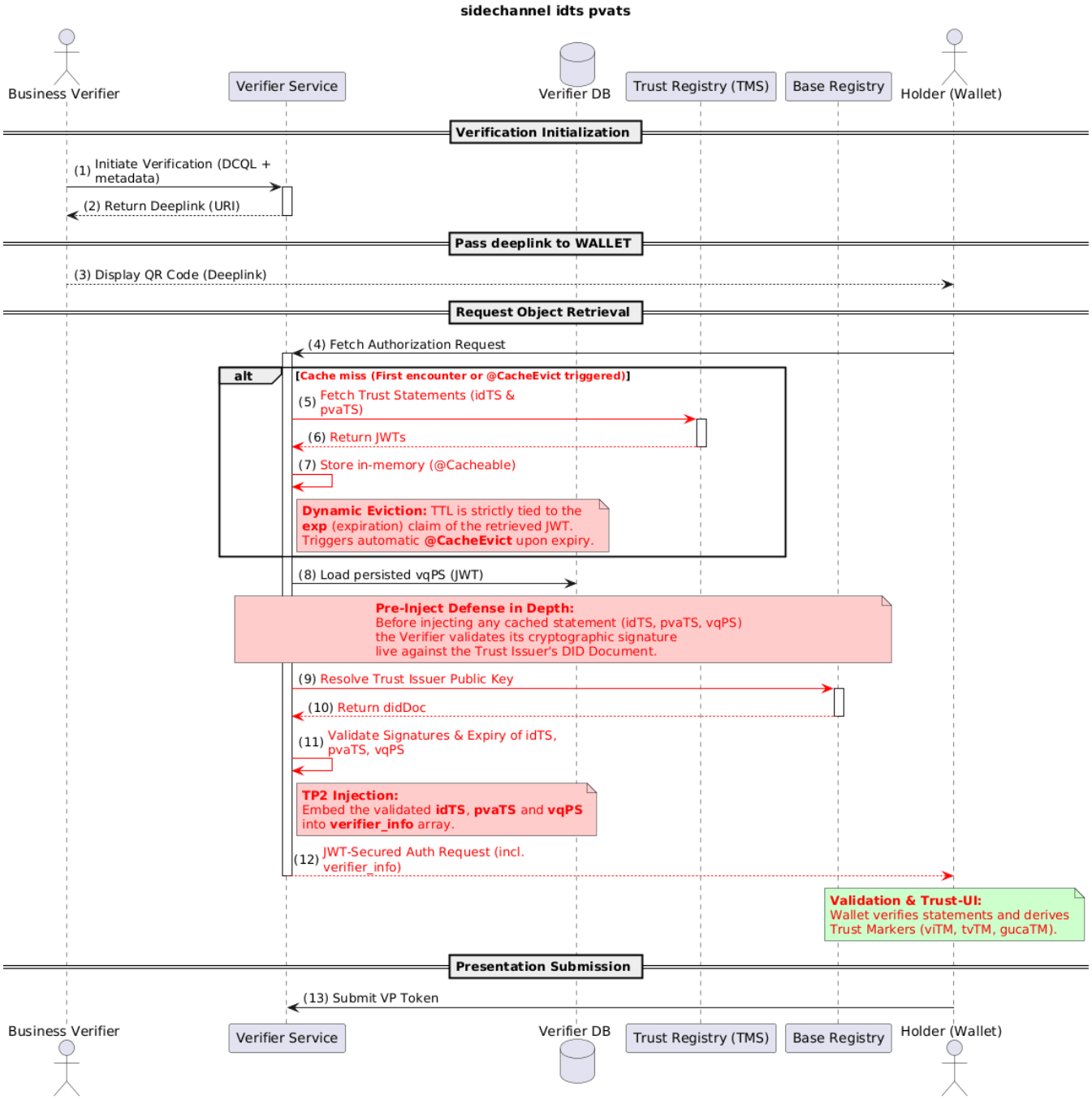


15.1. On-the-Fly Registration Flow

To minimize administrative overhead, the Generic Verifier fully automates the vqPS registration dynamically during the creation of a verification request. The architecture ensures cluster-safe execution and determinism via canonical hashing:

1. Request Creation: The Business Verifier initiates a verification by providing the dcql_query alongside mandatory transparency metadata, which includes a unique scope, a localized purpose_name, and a purpose_description.
2. Canonical Hashing & Cache Lookup: The Verifier Service computes a deterministic SHA-256 hash over the combination of the DCQL query and the localized metadata. This hash acts as the primary key to look up an existing vqPS in the database.
3. TMS Registration & Polling (If Unknown): If no valid vqPS is found for this specific hash, the service retrieves an OAuth2 Access Token (using a cluster-safe, distributed lock) and registers the query at the Trust Management System (TMS). The service reactively polls the TMS until the status reaches PUBLICATION_SUCCEEDED.
4. Storage & Caching: The Trust Service issues and returns the signed vqPS JWT. The Verifier Service persists this token in its database, keyed by the query hash, for immediate and future reuse.
5. Wallet Presentation: When the Wallet retrieves the Authorization Request, the Verifier Service fetches the cached vqPS JWT, performs a live security validation, and attaches it to the request payload.

16. Runntime - Sidechannel TS in the Generic Verifier



For the swiyu Wallet to establish a trusted relationship with the Verifier and render the Trust-UI, it must receive the Verifier's Trust Statements directly within the JWT-Secured Authorization Request (OID4VP Request Object).

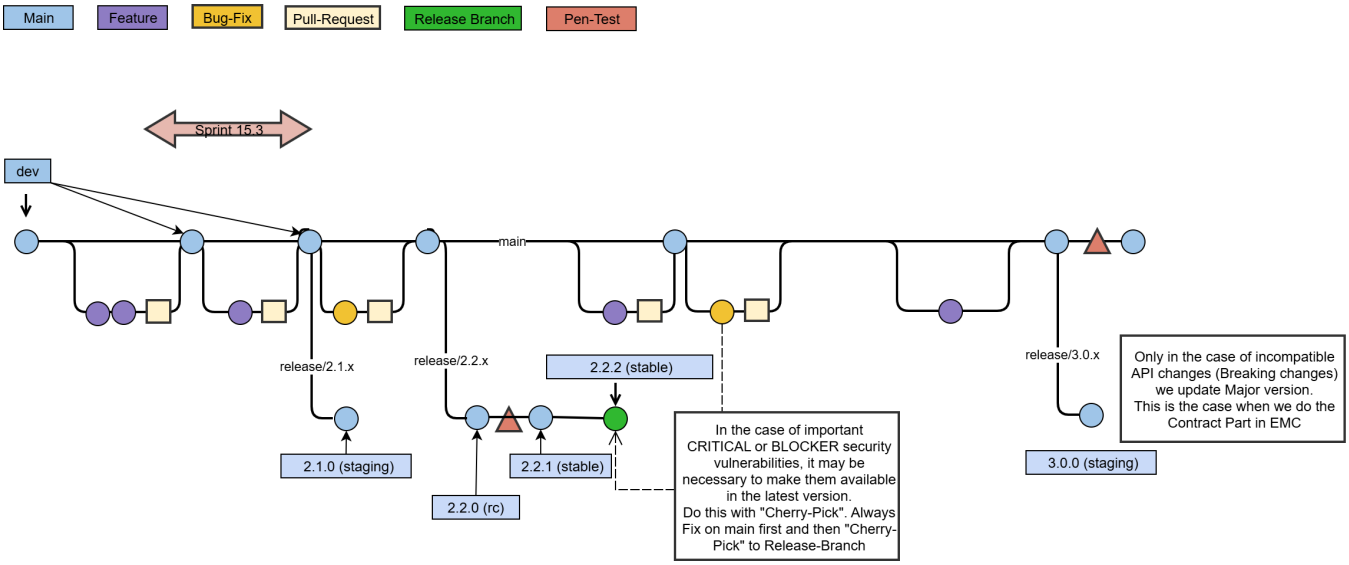
Because the Trust Statements have different lifecycles, the Generic Verifier utilizes a hybrid retrieval and caching architecture:

1. Identity & Authorization Statements (In-Memory Cache) The Identity Trust Statement (idTS) and the Protected Verification Authorization Trust Statement (pvaTS) are tied to the Verifier's organizational identity and general permissions. The Verifier fetches these from the Trust Management System (TMS) API on-demand during the first metadata request and caches them in-memory (e.g., using Spring's @Cacheable). The cache Eviction (TTL) is strictly tied to the exp (expiration) claim of the respective JWT.

2. The Verification Query Public Statement (vqPS) is tied to the specific DCQL query. As documented in the 6.2.8 Self-Service Portal for Verifiers to Register Verification Queries, the Verifier dynamically registers unknown DCQL queries with the TMS during the creation of a verification request (Initiate Verification). The returned vqPS JWT is persisted in the Verifier's database alongside the verification context for immediate and future reuse.

3. When the Holder scans the QR code and the Wallet fetches the Authorization Request, the Generic Verifier intercepts the payload generation. It reads the active idTS and pvaTS from the in-memory cache, and retrieves the matching vqPS from the database.
   It then dynamically embeds all three statements into the verifier_info array claim of the Request Object. Each statement is formatted strictly as {"format": "jwt", "data": "<jwt-string>"} without a credential_ids claim.

17. Crosscutting - Branches / Releases Concept



17.1. Naming Conventions of Branches

Type	Pattern	Example
Feature-Branch	feature/<JIRA-NR>_<JIRA-SUMARY>	feature/EIDOMNI-254_Some_Description_of_bla_bla
Bug-Fix-Branch	bugfix/<JIRA-NR>_<JIRA-SUMARY>	bugfix/EIDOMNI-123_Some_Description_of_bla_bla
Release-Branch	release/<RELEASE_NUMBER>	release/2.1.x

17.2. Semantic Versioning (SemVer) Overview

Semantic Versioning (SemVer) is a versioning scheme that makes it easy to understand the impact of a software release just by looking at the version number. A SemVer version number follows the format:

```
MAJOR.MINOR.PATCH
```

- **MAJOR version (X.y.z)**
  Incremented when we **Contract** the system by removing, changing, or breaking existing features.
  - Example: Removing a deprecated endpoint, changing response formats in a non-compatible way.
- **MINOR version (x.Y.z)**
  Incremented when we **Extend** the system with new, backward-compatible functionality.
  - Example: Adding a new endpoint, introducing an optional field, or extending valid inputs.
- **PATCH version (x.y.Z)**
  Incremented when we **Maintain** the system with backward-compatible security fixes on Release Branch.
  - Example: Security Bug fixes or important performance optimizations where are also needed on last Release

- **Security fixes for the release branch must always originate from the main branch.**

This system provides a predictable contract between maintainers and users: if you upgrade within the same MAJOR version, your existing integrations should continue to work. (Step Expand and Migrate in EMC Pattern)

17.2.1. Security Fix Process on release branch

> Policy: Upstream First ("Main First")
> As a non-negotiable rule, all bug fixes and security patches must first be developed and integrated on the main branch.
> Exception: If a fix must necessarily be developed directly on a release branch due to strongly diverging code bases, a mandatory "forward-port" ticket for the main branch must simultaneously be included in the current sprint.

1. Resolve the bug or security fix on the **main** branch.
2. **Cherry-pick** the commit onto the relevant release branch.

```
git checkout release/2.1.x
git cherry-pick <commit-hash>
```

3. Run tests and verify the fix on the release branch.
4. Publish a new release with a **PATCH version bump** (e.g., `2.1.0` `2.1.1`).
5. Document the fix in the release notes.

17.3. Releases

Docker images for this project follow a formalized environment-based tagging approach:

Tag	Meaning	Description
dev	Development Build	Latest commit from the development branch. Automatically generated on every push to `main`.
staging	Integration Test Build	Set at the end of a sprint or after completion of a feature for integration testing.
rc	Release Candidate	Frozen state prior to release and penetration testing.
stable	Verified Production	Released after successful QA and penetration testing.

These tags are assigned automatically or manually as part of the CI/CD workflow. This ensures that environments can reliably reference images by their lifecycle stage (e.g., `swiyu-issuer-service:staging`) without requiring manual version management.

17.3.1. Promotion Workflow

The image promotion process follows these steps:

```
[Commit  dev]
        build & push :dev
[Feature completed / Sprint end]
        promote  :staging
[Release candidate created]
        promote  :rc
[QA & penetration test passed]
        promote  :stable
```

17.3.2. When to create a official release

Before any offinial release can be approved, a **penetration test must be conducted**.
A release may only be made if the penetration test has been successfully completed and accepted.

- **When penetration tests are required**
    - 01. Pentest - Policy Process according to 1.5. Pentesting Decision
    - Typically every few months
    - When introducing new features
    - When making changes or adjustments to the API
- **After the tested release branch**
  On a release branch that has already passed a penetration test, additional **non-pen-test-critical security fix patches** may be integrated. These can be included in subsequent releases **without requiring another penetration test**.